

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY :

Software Engineering

MES-X Dependency Management

Traceability and Release-Readiness Analysis on Azure DevOps

Prepared by

Alaa Eddine Mdalla

Internship carried out at

Forvia Informatique Tunisie

Academic Year
2024/2025

Acknowledgments

This work would not have been possible without the support, guidance, and kindness of the many people who accompanied me throughout this journey.

First and foremost, I would like to express my sincere gratitude to **Ms. Khemiri Karima**, **Abassi Chifa** and **Ms. Gabsi Oumeima** for their constant support, thoughtful guidance, and genuine encouragement throughout my internship at Forvia. Their professionalism, trust, and kindness made this experience both enriching and memorable, and I remain deeply thankful for the confidence they placed in me from the very beginning.

I am equally grateful to **Ms. Boumaiza Wafa** for her attentive academic supervision and the valuable advice she provided at every stage of this work. Her patience, rigor, and continuous encouragement were a constant source of motivation and played an essential role in shaping this project.

I would also like to extend my heartfelt thanks to the **MES-X team** for their collaboration, openness, and the welcoming environment they created throughout this experience. I am especially grateful to **GHAZOUANI Mohamed Neji**, **BEJAOUI Nawres**, **SALAH Houssam**, and **YOUAZ Ichrak**, whose support, availability, and team spirit contributed greatly to both my professional growth and the success of this internship.

Beyond the professional sphere, I am profoundly grateful to the people who have been my pillars throughout this journey.

My first and most heartfelt thought goes to my **late father, Sammi Mdalla**, who passed away last March. He did not have the chance to see this work completed, but his presence lives on in every page of it. His sacrifices, his values, and his unwavering belief in me are the foundation upon which everything I have achieved has been built. I carry his memory with me always.

To my **mother, Hedia Ben Jouda**, thank you for your boundless love, your patience, and your strength. You have always been my refuge and my source of courage. To my **brother, Dhia Mdalla**, and my **sister, Wafa Mdalla**, thank you for your affection, your presence, and for always believing in me, even in moments when I doubted myself.

This accomplishment belongs to all of you.

Abstract

Large software teams working with Azure DevOps need a reliable way to trace the links between work items, pull requests, commits, branches, and release tags. Within the MES X.0 environment at Forvia, this traceability was previously built through a manual process that was slow, repetitive, and error-prone.

The **MES-X Dependency Management** project addresses this limitation by providing a read-only traceability service composed of a NestJS backend and a Vue 3 frontend. The backend integrates with Azure DevOps, traverses dependency graphs, resolves pull requests and commits, streams batch results, and identifies the tag context of each change. The frontend gives users an operational interface for launching analyses, following progress, and reviewing consolidated results.

This report presents the project context, the formal requirements, the proposed architecture, the implementation choices made for both backend and frontend, and the main technical outcomes of the internship.

Keywords: NestJS, Azure DevOps, Vue 3, dependency management, traceability, internship report.

List of Figures

1.1	Azure DevOps environment used within the MES X.0 division, showing work item tracking and development workflow management	3
1.2	Excerpt from the version.json file showing microservice versions across DEV, QUA, PREPROD, and PROD environments	5
2.1	Manual dependency tracking workflow before the MES-X Dependency Management system	14
3.1	Use Case Diagram of the MES-X Dependency Management System. . .	24
4.1	Global connection graph of the Trace UI and backend modules	30
5.1	Frontend technology used: Vue 3 for building the user interface	37
5.2	Backend technology used: NestJS for scalable server-side architecture .	37
5.3	Azure DevOps Platform	39
5.4	Agile Scrum methodology applied during system development	39
5.5	Backend class diagram for the work item, pull request, and commit modules	45
5.6	Backend class diagram for the sprint and decision modules	45
5.7	Sequence diagram for single-root related work-item processing	48
5.8	Sequence diagram for batch related work-item aggregation	49
5.9	UI route and component architecture for traceability workflows	51
5.10	Sequence diagram for UI and backend integration in trace workflows . .	52
5.11	Batching Work Items – selection and SSE-driven progress	53
5.12	Related Work Item – single-root traceability graph view	54

5.13	Discovered Commits – commits found during traversal	55
5.14	Decision – work items grouped by state (Done vs. Other)	55
5.15	Decision – repository view with expected tags for promotion	56

List of Tables

3.1	Functional Requirements to System Component Traceability Matrix . .	25
-----	---	----

List of Acronyms

Abbreviation	Definition
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
DTO	Data Transfer Object
DTOs	Data Transfer Objects
E2E	End-to-End
Epic	High-level Azure DevOps work item used to group major business capabilities
FR	Functional Requirement
Git	Distributed version control system used by Azure DevOps repositories
MES	Manufacturing Execution System
MES-X	Forvia's Manufacturing Execution System platform context for this dependency-management service
NFR	Non-Functional Requirement
NestJS	Node.js framework used to build the backend
PAT	Personal Access Token
PR	Pull Request

Abbreviation	Definition
Quasar	Vue-based UI framework used to build the frontend interface
REST	Representational State Transfer
SSE	Server-Sent Events
TypeScript	Typed superset of JavaScript used in both backend and frontend development
UI	User Interface
UML	Unified Modeling Language
UC	Use Case
Vite	Frontend build tool used to bundle and serve the Vue application
Vue	Progressive JavaScript framework used for the frontend application

General Introduction

Managing a large software platform is never only a matter of writing code. In the MES X.0 environment at Forvia Informatique Tunisie, teams work on a microservices-based Manufacturing Execution System deployed across several industrial sites. Azure DevOps supports this work by centralizing planning, source control, code review, and pipeline automation.

The real difficulty lies in understanding how all of these development artifacts relate to one another. A single feature may involve several work items, multiple pull requests, and many commits spread across different repositories. Before this project, tracing those links required a largely manual process based on Azure DevOps navigation and spreadsheet tracking. The approach was slow, repetitive, and difficult to reproduce reliably.

This internship addressed that problem by designing and implementing an automated traceability system able to reconstruct dependency chains directly from Azure DevOps data. The objective was not simply to save time, but to give developers and release engineers a clearer and more dependable view of the relationships between requirements, code changes, and release tags.

The rest of this report follows the progression of that work. It first presents the industrial context and the project objectives, then formalizes the system requirements, explains the proposed architecture, and finally describes the implementation choices and the main results achieved during the internship.

Chapter 1

Project Overview

1.1 Introduction

This chapter establishes the professional and technical context of the end-of-study internship project. It presents the host company, the industrial environment in which the work was carried out, the problem that motivated the project, and the main objectives assigned during the internship.

This context is important because the technical decisions described in the next chapters were shaped by concrete industrial constraints: a large microservices landscape, a growing number of Azure DevOps artifacts, and the operational need for faster and more reliable traceability.

1.2 Host Company

1.2.1 The Forvia Group

Forvia is one of the world's leading automotive technology groups, formed through the strategic merger of two major European industrial players: **Faurecia**, a global specialist in automotive seating, interiors, and emissions control technologies, and **HELLA**, a globally recognized leader in automotive lighting systems and electronic components. The merger created a group of exceptional scale, combining complementary technological portfolios and international footprints to serve automotive manufacturers across every major global market.

Forvia operates across more than 40 countries, employs over 150,000 people worldwide, and maintains a strong research and development focus aimed at accelerating

the transition toward sustainable, intelligent, and connected vehicles. Its product and technology portfolio spans seating systems, cockpit electronics, clean mobility solutions, lighting, and sensors — placing the group at the intersection of several of the most critical transformation vectors in the modern automotive industry.

1.2.2 Forvia Informatique Tunisie

The internship took place at **Forvia Informatique Tunisie**, the Tunisian IT and digital engineering hub of the Forvia Group. Established as a center of technical excellence within the group’s global digital organization, this entity is responsible for the design, development, and long-term maintenance of the software systems that support industrial operations at Forvia’s manufacturing facilities worldwide.

Forvia Informatique Tunisie concentrates a multidisciplinary team of software engineers, architects, DevOps specialists, and technical leads who work in close collaboration with operational and product teams across multiple Forvia business units. The entity contributes to a broad range of digital initiatives, including manufacturing execution systems, production monitoring platforms, enterprise integration solutions, and internal tooling designed to improve the efficiency and traceability of the group’s software delivery processes.

1.2.3 The MES X.0 Division

The project was done in the MES X.0 division of Forvia Informatique Tunisie. This team works on building and improving a modern Manufacturing Execution System. This system is used in factories to manage, control, and track production in real time.

The MES X.0 platform uses a microservices architecture. is composed of several small services, each responsible for a specific role, such as production planning, quality control, equipment monitoring, or helping operators on the shop floor. These services communicate with each other using APIs.

This design makes the system more flexible. Each service can be developed and updated separately without affecting the whole system. It also allows each factory to use only the parts it needs.

Today, the MES X.0 platform is used in several Forvia factories around the world and processes real-time data from machines and operators.

However, this architecture is more complex to develop and maintain. This complexity is what led to the work presented in this report.

1.3 Project Context and Problem Statement

1.3.1 Development Environment at Forvia

At Forvia, **Azure DevOps** serves as the central platform for the software development lifecycle across all digital teams. It provides an integrated suite of tools covering the full range of development activities: sprint and release planning through work item boards, source code management through Git repositories, code review and integration through pull requests, automated build and deployment pipelines through Azure Pipelines, and artifact management for binaries and package versions.

Within the MES X.0 division specifically, Azure DevOps manages the development activities of all microservices simultaneously. A given sprint may involve planned work across ten, twenty, or more independent services, each progressing at its own pace with its own team, its own repository, and its own release cadence. The platform accumulates an ever-growing set of interconnected artifacts: work items describing requirements and tasks, pull requests integrating code changes, commits recording individual code modifications, branches isolating development streams, and repository tags marking release points.

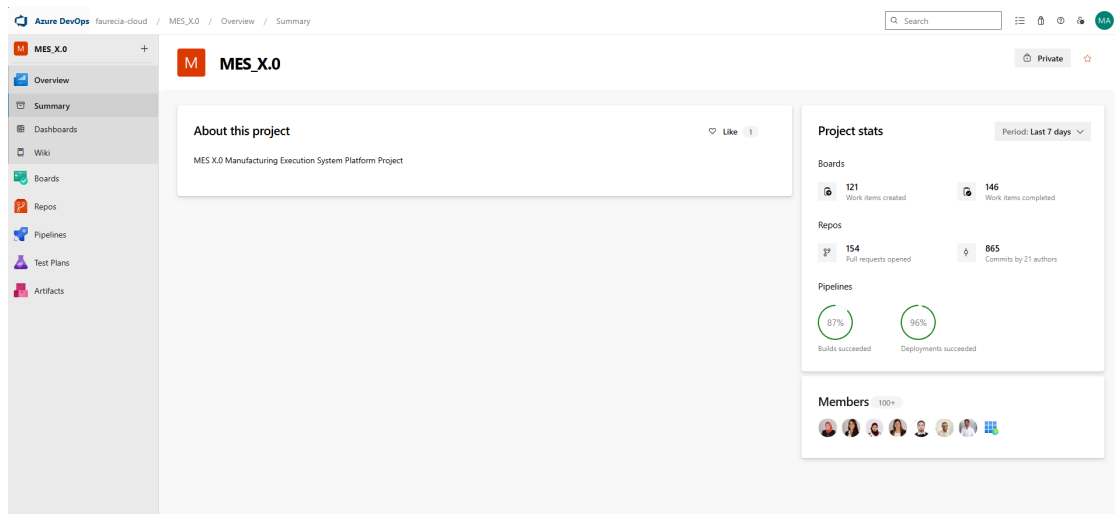


Figure 1.1. Azure DevOps environment used within the MES X.0 division, showing work item tracking and development workflow management

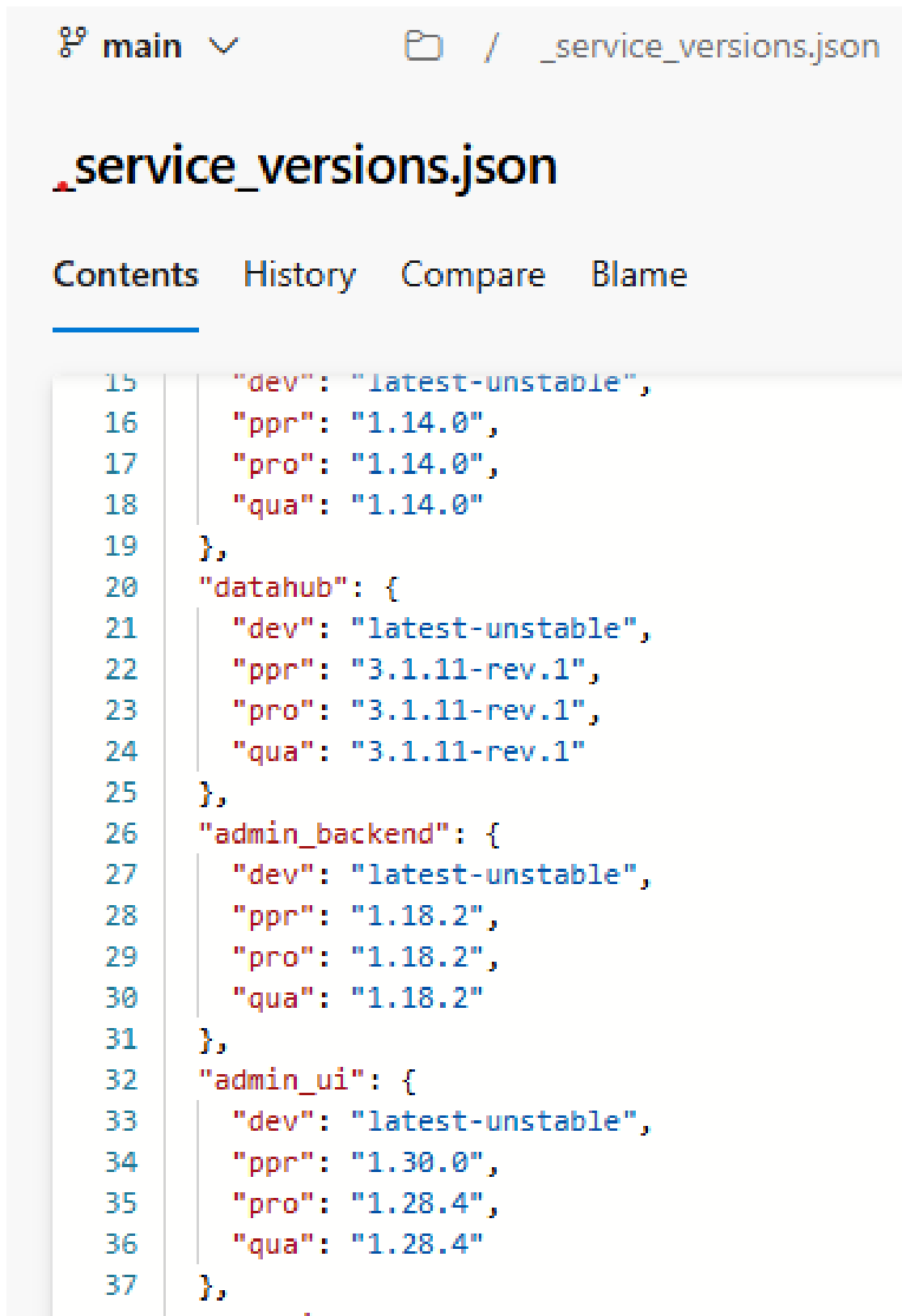
1.3.2 Deployment Environments and Versioning Strategy

In the MES X.0 ecosystem, each microservice is deployed across several environments that represent the standard software delivery lifecycle. These environments include **Development (DEV)**, **Quality Assurance (QUA)**, **Pre-production (PREPROD)**, and **Production (PROD)**.

Each environment plays a specific role in the validation and release process. The DEV environment is used for implementing and integrating new features. The QUA and PREPROD environments are used to validate functionality under conditions that are close to production, while the PROD environment hosts the live system used by manufacturing sites.

A key characteristic of the platform is that each microservice follows its own versioning across these environments. As a result, a service may not necessarily run the same version in DEV, QUA, PREPROD, and PROD at the same time, depending on its deployment and release cycle.

This versioning information is centralized in the `version.json` file located in the **infra repository**. This file defines the mapping between microservices and their deployed versions in each environment.



```
15     "dev": "latest-unstable",
16     "ppr": "1.14.0",
17     "pro": "1.14.0",
18     "qua": "1.14.0"
19 },
20 "datahub": {
21     "dev": "latest-unstable",
22     "ppr": "3.1.11-rev.1",
23     "pro": "3.1.11-rev.1",
24     "qua": "3.1.11-rev.1"
25 },
26 "admin_backend": {
27     "dev": "latest-unstable",
28     "ppr": "1.18.2",
29     "pro": "1.18.2",
30     "qua": "1.18.2"
31 },
32 "admin_ui": {
33     "dev": "latest-unstable",
34     "ppr": "1.30.0",
35     "pro": "1.28.4",
36     "qua": "1.28.4"
37 },
```

Figure 1.2. Excerpt from the version.json file showing microservice versions across DEV, QUA, PREPROD, and PROD environments

This file is essential for understanding the real state of the system at any given time. It is especially important during release analysis and dependency tracking, as it

helps identify which version of each service was active in each environment.

However, this environment-based versioning model also introduces additional complexity when analyzing system changes across services, particularly when trying to trace dependencies between work items and deployed versions.

1.3.3 The Traceability Challenge

In this multi-service development environment, a single user requirement, usually defined as an Epic or Feature in Azure DevOps, is not implemented in one place. It is broken down into smaller tasks that are distributed across different teams and services. Each task produces its own development artifacts such as branches, commits, pull requests, and finally a release tag when the change is delivered.

Because of this structure, understanding the full impact of a requirement, or identifying which requirements are affected by a change in a specific service, requires following the complete chain of artifacts.

For a release engineer preparing a production deployment, this often means answering questions such as:

- Which work items are included in this release, and are they completed?
- For each work item, which pull requests were used and in which repositories?
- For each repository, which commits were made, and do they belong before or after the release tag?
- Are there commits that are part of one release but not yet included in another environment?

Before this project, answering these questions was a manual process. A release engineer would start from a work item in Azure DevOps, follow its linked pull requests, open each one to identify the repository, then check the commit history to find the related changes. This had to be repeated for every work item, and the information was usually stored in spreadsheets for tracking.

This process had several limitations:

- **Time-consuming:** tracing a single feature across multiple services could take a full working day.
- **Error-prone:** manually linking information across Azure DevOps pages increased the risk of missing or incorrect data.

- **Inconsistent:** different engineers could follow different paths and get different results.
- **Not scalable:** as the number of microservices increased, the process became too slow for regular release cycles.

1.3.4 Project Motivation

These limitations created a clear and urgent need for an automated solution. The **MES-X Dependency Management** project was initiated within the MES X.0 division to address this need directly: to design and implement a system that would automate the reconstruction of dependency chains across Azure DevOps artifacts, transforming a time-consuming and error-prone manual process into a fast, reliable, and reproducible analytical capability.

The project was scoped as an end-of-study internship project, representing an opportunity to deliver genuine industrial value while applying and extending the software engineering knowledge acquired during the academic program.

1.4 Project Description

1.4.1 Project Objectives

The primary goal of the MES-X Dependency Management project is to design and implement an **automated dependency traceability system** that replaces the existing manual process with a reliable, efficient, and scalable solution integrated into the Azure DevOps ecosystem.

The project pursues the following specific objectives:

Automated Microservice Tag Identification

The system must automatically identify and associate the relevant microservice repository tags with work items by analyzing commit history and repository metadata. This capability eliminates the need for manual navigation through repository tag lists and commit histories, and ensures that the release context of each code change is determined programmatically and consistently.

Comprehensive Dependency Mapping

The system should be able to build and provide clear traceability links across the whole chain of development artifacts, starting from high-level work items and going through pull requests and commits, down to branches and repository tags. This mapping should cover all levels of the work item hierarchy and all types of artifacts, and it should remain consistent each time the analysis is run.

Significant Reduction of Manual Effort

By automating data retrieval, traversal, normalization, and aggregation, the system must reduce the time required to perform a complete dependency analysis from hours to seconds. This reduction must be sustained even for complex analysis scenarios involving multiple work items and many repositories.

Improved Data Accuracy and Consistency

The system should remove errors caused by manual data extraction and linking. All dependency data should come directly from a single reliable source, which is the Azure DevOps APIs, using a standard and repeatable process. This ensures that the results are accurate and remain the same every time the analysis is performed with the same input.

End-to-End Traceability

The system should allow a user to trace any work item from its initial description down to the exact commits and microservice tags it impacted, all within a single analysis session. The user should not need to switch between tools or manually connect information to get a complete view of the dependencies.

Reusable and Scalable Architecture

The system should be designed with a modular and maintainable architecture. It should be easy to extend in the future to support new types of artifacts, new analysis features, or integration with other Forvia platforms, without having to redesign the whole system.

1.4.2 Technical Scope

The project includes the design and development of three main parts: a backend service, a frontend application, and an integration layer with Azure DevOps.

Backend Development

Backend Development

The backend was developed using **NestJS** with TypeScript. This framework was chosen because it supports modular design and helps keep the code clean and easy to maintain.

The backend is responsible for:

- Connecting securely to Azure DevOps using Personal Access Tokens.
- Transforming raw data from Azure DevOps into structured and typed data used inside the system.
- Traversing work item dependencies across different levels.
- Retrieving related pull requests, commits, branches, and tags.
- Supporting analysis starting from different entry points such as a single work item, multiple work items, or entire sprints.
- Providing a Sprint module that allows retrieving all work items related to one or more sprints, as well as their associated repository versions.
- Integrating an LLM-based module to review commits based on MES patterns and predefined rules, in order to verify that changes follow the expected development standards.
- Reducing the number of API calls using batch processing.
- Streaming results progressively using Server-Sent Events.
- Providing clear REST APIs with Swagger documentation for testing.

Frontend Application

The frontend was built using **Vue 3** with the Quasar framework. It provides a responsive interface for running and exploring dependency analysis.

The frontend is responsible for interacting with the backend and displaying results in a clear and structured way, including real-time updates during the analysis process.

Main features include:

- Visualization of work item dependencies in a clear and structured way.
- Support for different analysis entry points, such as a single work item, multiple work items, or entire sprints.
- Sprint-based views that allow users to explore all work items linked to one or more sprints, along with their related repository versions.
- Real-time progress updates using Server-Sent Events.
- Display of relationships between work items, pull requests, commits, branches, and tags.
- A Decision Dashboard that summarizes release and dependency results.
- A dedicated view for LLM-based commit review results, showing whether commits respect MES patterns and coding guidelines.
- A simple and practical design focused on daily use by engineers and release teams.

Azure DevOps Integration and Data Processing

The system uses Azure DevOps REST APIs as its only data source. This integration layer handles all communication with Azure DevOps and ensures that the data is retrieved correctly and efficiently.

It includes:

- Secure authentication using Personal Access Tokens.
- Batch processing and control of API requests to avoid rate limits.
- A dependency engine that explores relationships and removes duplicates.
- Error handling and retry mechanisms to avoid incomplete results.

1.4.3 Project Activities

The project followed several main steps:

- **Requirements Analysis:** understanding the current manual process and defining system needs with engineers.
- **System Design:** defining the architecture, backend components, and data models.
- **Backend Development:** building the integration with Azure DevOps and implementing the dependency analysis logic.
- **Frontend Development:** creating the user interface and integrating real-time updates.
- **Testing:** verifying that the system works correctly and meets requirements.
- **Optimization:** improving performance and ensuring consistent results.
- **Deployment Preparation:** preparing Docker setup, configuration, and documentation.

1.5 Conclusion

This chapter presented the project context and explained why dependency traceability became a real issue. In a microservices environment, manual analysis is no longer efficient or reliable for release preparation.

The next chapter will define the system requirements and describe what the solution must achieve.

Chapter 2

Study of the Existing

2.1 Current Dependency Tracking Process

This chapter analyzes the manual workflow used prior to the introduction of the MES-X Dependency Management system. Understanding this existing process is essential, as the limitations identified here directly influence the design of the proposed solution.

In the MES X.0 environment, the process typically begins with a **user story** created in Azure DevOps. Each user story is often linked to multiple child tasks. In turn, each task may generate its own pull requests, commits, and release references across different repositories.

Although Azure DevOps stores all these artifacts, it does not provide a unified or consolidated view of their dependency relationships. As a result, developers must manually reconstruct the dependency chain.

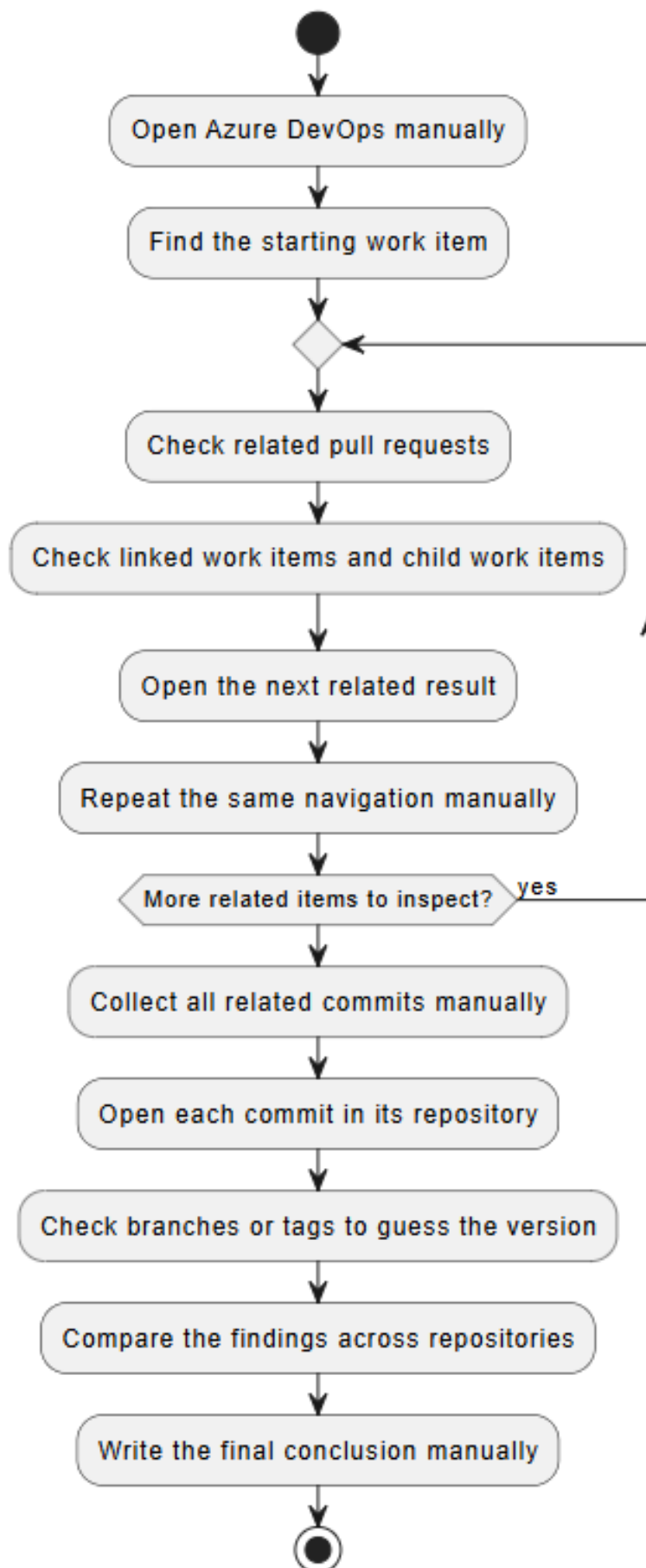
Manual Dependency Analysis Workflow

In practice, developers preparing a dependency report must follow a repetitive sequence of steps for each user story:

1. Open the work item in Azure DevOps and identify all linked child tasks and related items.
2. Navigate to the **Development** panel of each item to locate associated pull requests.
3. Open each pull request individually and access its **Commits** tab.
4. Analyze commit messages and modified files to determine the impacted microservice and extract version tags.

5. Manually record all relevant data (work item ID, pull request number, commit hash, microservice name, tag) in a spreadsheet.
6. Repeat the same process for every child work item linked to the main user story.

During a typical sprint, which may include multiple user stories and several pull requests per story, this process can consume a significant portion of a working day. Each dependency must be manually identified, verified, and recorded. As the number of services and work items increases, this manual approach becomes increasingly time-consuming and repetitive.

Dependency Management Flow Before This Project

2.2 Illustrative Example

Consider a user story such as “*Update shared configuration service for a specific environment.*” This change may involve multiple components, including the configuration service, a dependent API gateway, and deployment adjustments.

In practice, this single user story is divided into several child tasks, each associated with its own pull request and set of commits.

To construct a complete dependency view, the developer must:

- Inspect each pull request,
- Identify the affected repository,
- Review associated commits,
- Extract and compare release tags.

Additionally, commit tags must be analyzed to determine the most recent release version. It is also necessary to verify whether commits belong to a tagged release or fall between two versions.

Even in relatively simple scenarios, this requires navigating multiple pull requests and analyzing numerous commits. When repeated across an entire sprint, the workload becomes substantial and error-prone.

2.3 Limitations and Their Impact

The manual dependency tracking process introduces several key challenges:

Time Consumption Generating a complete dependency report for a sprint can take several hours or even an entire day. Most of this time is spent navigating tools and manually collecting data rather than focusing on development tasks.

High Risk of Errors The process relies entirely on manual verification. Missing a commit, overlooking a repository, or misidentifying a tag can result in incomplete or incorrect dependency reports, potentially impacting release decisions.

Limited Scalability The MES X.0 platform consists of numerous microservices, with new ones being added continuously. As the system grows, the manual approach becomes increasingly difficult to maintain and scale.

Lack of a Single Source of Truth Dependency data is typically stored in spreadsheets, leading to inconsistencies. Different engineers may produce different results for the same feature, with no centralized mechanism to validate or reconcile discrepancies.

Overall, these limitations negatively impact both efficiency and reliability. They clearly highlight the need for an automated solution, which will be introduced in the next chapter through the system requirements.

Chapter 3

Requirements Specification

3.1 Introduction

This chapter describes what the MES-X Dependency Management system must do and how well it must do it. The goal at this stage is to describe the expected behavior independently of implementation choices, so the architecture and code can later be measured against a clear baseline.

The requirements come from studying the manual workflow described in the previous chapter, talking with developers and release engineers, and watching traceability tasks being carried out in the MES X.0 environment. They cover both what the system must do and the quality bar it must meet in daily use.

3.2 Functional Requirements

The functional requirements define the main capabilities of the MES-X Dependency Management system. They describe how the system processes Azure DevOps data to build a complete traceability view across work items, source code, and release information.

- **FR-01 – Pull Request Resolution**

The system shall retrieve pull requests from Azure DevOps and associate them with their corresponding work items. This enables the system to establish a consistent link between work tracking elements and source control changes, which forms the entry point of the traceability pipeline.

- **FR-02 – Commit and Tag Contextualization**

The system shall extract commits linked to each pull request and add contextual

information, including branch details and repository tags. This contextual information is required to determine the position of each change within the release lifecycle and to support release-level analysis.

- **FR-03 – Dependency Graph Construction**

The system shall construct a dependency graph starting from one or multiple root work items. It shall traverse all related entities through Azure DevOps relationships, including parent-child links, pull request associations, and commit references, in order to reconstruct the full dependency chain.

- **FR-04 – Aggregation and Decision Output**

The system shall aggregate all retrieved entities into a unified data model. This includes normalization of work items, pull requests, and commits, as well as removal of duplicates caused by multi-path traversal. The final output shall provide a structured view that supports release analysis and decision-making.

- **FR-05 – End-to-End Read-Only Traceability**

The system shall provide a complete traceability view across all relevant artifact levels: work items, pull requests, commits, branches, and tags. All interactions with Azure DevOps shall be strictly read-only, ensuring that no modification, creation, or deletion of external data is performed by the system.

- **FR-06 – Sprint-Based and Multi-Sprint Dependency Resolution**

The system shall support dependency analysis starting from one or more sprints, as well as from work items belonging to multiple sprints. It shall retrieve all related work items and trace their dependencies across pull requests, commits, and repository versions.

The system shall also identify the corresponding release versions associated with these work items in order to support validation of transitions between environments (e.g., development, pre-production, production). This enables the system to determine which changes are required to move a set of work items to the next environment.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes that the system must respect to be usable in an industrial MES X.0 environment. They ensure that the system is not only functionally correct, but also efficient, secure, maintainable, and scalable in a distributed microservices context.

3.4 Performance

Performance requirements focus on ensuring that dependency analysis remains efficient even in a large and interconnected microservices ecosystem.

- **NFR-PE-01 – Efficient Graph Traversal:** The system must rely on optimized graph traversal strategies such as BFS and DFS to explore work item relationships, ensuring controlled and complete dependency resolution without redundant processing.
- **NFR-PE-02 – Batch Processing Strategy:** The system must reduce external API overhead by using batching techniques when interacting with Azure DevOps, grouping requests where possible and avoiding repeated calls for the same entities.
- **NFR-PE-03 – Caching and Deduplication:** Intermediate results (work items, pull requests, commits) must be cached and deduplicated during traversal to avoid repeated computation and improve response efficiency.
- **NFR-PE-04 – Streaming Feedback:** Long-running analysis operations must provide progressive feedback to the user through streaming updates to maintain usability and responsiveness.

3.5 Scalability

Scalability ensures that the system remains stable as the number of work items, repositories, and services grows.

- **NFR-SC-01 – Algorithmic Scalability:** The use of BFS/DFS-based traversal combined with caching ensures that the system scales with graph complexity rather than naive repeated exploration.
- **NFR-SC-02 – Microservice Modularity:** The architecture follows a strict separation of concerns aligned with SOLID principles, where each service is responsible for a single domain (work items, PRs, commits, aggregation), enabling independent evolution.
- **NFR-SC-03 – MES Pattern Compliance:** The system design follows MES-oriented architectural patterns used in the platform, ensuring that new services can be integrated without modifying existing ones.

- **NFR-SC-04 – Dapr-Based Service Expansion:** The system uses Dapr as a distributed application runtime to support service discovery, state management, and secret handling, allowing horizontal scalability across microservices.

3.6 Reliability

Reliability ensures that results remain consistent and trustworthy in all execution scenarios.

- **NFR-RE-01 – Deterministic Execution:** Given the same input and system state, the system must always produce consistent results regardless of execution order or concurrency.
- **NFR-RE-02 – Fault Isolation:** Failures in external services (e.g., Azure DevOps API) must be isolated to prevent corruption of the full analysis pipeline.
- **NFR-RE-03 – Controlled Recovery:** The system must implement retry mechanisms for transient failures while preserving partial results.
- **NFR-RE-04 – Data Consistency:** All entities (work items, commits, PRs, tags) must remain consistent across all processing stages.

3.7 Maintainability

Maintainability ensures that the system remains easy to evolve and extend over time.

- **NFR-MA-01 – Single Responsibility Design:** Each module must have a single clear responsibility, following SOLID principles and reducing coupling between components.
- **NFR-MA-02 – KISS Principle:** The system must favor simple and readable implementations over unnecessary complexity, especially in core traversal and aggregation logic.
- **NFR-MA-03 – Clean Separation of Concerns:** Business logic, API integration, and data aggregation must remain separated to improve testability and long-term maintainability.
- **NFR-MA-04 – Reusable Components:** Core services must be reusable across different MES modules without modification.

- **NFR-MA-05 – Standardized Code Structure:** Naming conventions, module organization, and API contracts must remain consistent across the system.

3.8 Security and Data Integrity

Security ensures controlled access to industrial data and prevents unauthorized usage of the system.

- **NFR-SE-01 – Token-Based Authentication:** Both backend and frontend must use Azure DevOps Personal Access Tokens (PAT) for secure API access.
- **NFR-SE-02 – Role-Based Access Control:** The frontend must restrict access to analysis features using authentication guards, ensuring that only authorized users can access developer-level functionality.
- **NFR-SE-03 – Secure Secret Management:** Sensitive credentials and tokens must be managed using Dapr secret store integration, avoiding hardcoded or exposed configuration values.
- **NFR-SE-04 – Controlled External Access:** All communication with Azure DevOps must be routed through the backend integration layer, preventing direct external access from the frontend.
- **NFR-SE-05 – Read-Only Enforcement:** The system must operate in strict read-only mode with respect to Azure DevOps, ensuring no modification of external artifacts.

3.9 Use Case Model

The use case model shows how the actors interact with the system to carry out traceability and analysis tasks. It complements the requirement lists with a behavioral view of the system.

3.9.1 Actors

The use case diagram highlights the two primary actors shown in the figure: a Developer and an LLM Provider. The diagram is intended to make the main workflow and intelligence flow visible at a glance.

Developer and Release Engineer

The primary user. Developers use the system to understand the impact of a work item on the codebase. Release engineers use it to check release scope and confirm whether changes sit on the correct side of a release boundary.

This actor starts analysis sessions, chooses the analysis mode, reviews results, and uses the Decision Dashboard output to support release, integration, and escalation decisions.

LLM Provider

An external AI service used by the system to review commits and generate intelligent insights. This actor supports the "Review Commit with LLM" use case and is shown in the diagram as a cross-cutting service.

Additional Actors

Other actors are part of the overall model but are not illustrated in this view. The Administrator manages operational configuration and availability, and Azure DevOps is the external data source for work items, pull requests, commits, branches, and tags.

3.9.2 Use Case Descriptions

The following use cases describe what the system does from each actor's perspective.

Work Item Use Cases

- **UC-01 – Analyze Single Work Item:** The user provides one work item ID. The system traverses the full dependency graph, resolves related pull requests and commits, enriches commits with branch and tag context, and returns a complete normalized trace result.
- **UC-02 – Analyze Multiple Work Items:** The user provides several work item IDs. The system traverses each root, merges and deduplicates the results, and returns one consolidated trace. This supports release scope analysis across a sprint or milestone.
- **UC-03 – Stream Work Item Tasks:** The user starts a batch analysis session. The system streams resolved work item tasks progressively so progress is visible

in real time, and items can be selected for further analysis before the full dataset has loaded.

Pull Request Use Cases

- **UC-04 – Retrieve Pull Request Work Items:** Given a pull request identifier, the system returns the work items formally linked to that PR on the Git side, along with the normalized PR summary including the repository name.

Commit Use Cases

- **UC-05 – Retrieve Commits by Work Item:** Given a work item ID, the system returns the deduplicated list of commits linked to that item, resolved through direct links and pull request expansion.
- **UC-06 – Retrieve Commits by Pull Request:** Given a pull request ID, the system returns the list of commits merged as part of that PR.
- **UC-07 – List Repository Branches and Tags:** Given a repository ID, the system returns the available branch and tag references needed to interpret commit positions.
- **UC-08 – Resolve Commit Tag Context:** Given a repository and a commit, the system determines the most recent tag reachable from that commit on the relevant branch.
- **UC-09 – Batch Resolve Commit Tag Context:** Given a repository and multiple commits, the system resolves the tag context of all specified commits in one operation to reduce API usage on large graphs.

Aggregation and Decision Use Cases

- **UC-10 – Generate Dependency View:** After a trace is complete, the system aggregates all collected data — work items, pull requests, commits, and tag context — into a single decision-ready view. Work items are grouped by completion state and repositories are classified by their commit position relative to the target release.
- **UC-11 – Configure System:** The Administrator updates environment-level configuration, including Azure DevOps connection settings, API credential management, and deployment parameters.

3.9.3 Use Case Diagram

Figure 3.1 shows the complete use case diagram, with the primary actors and all eleven use cases.

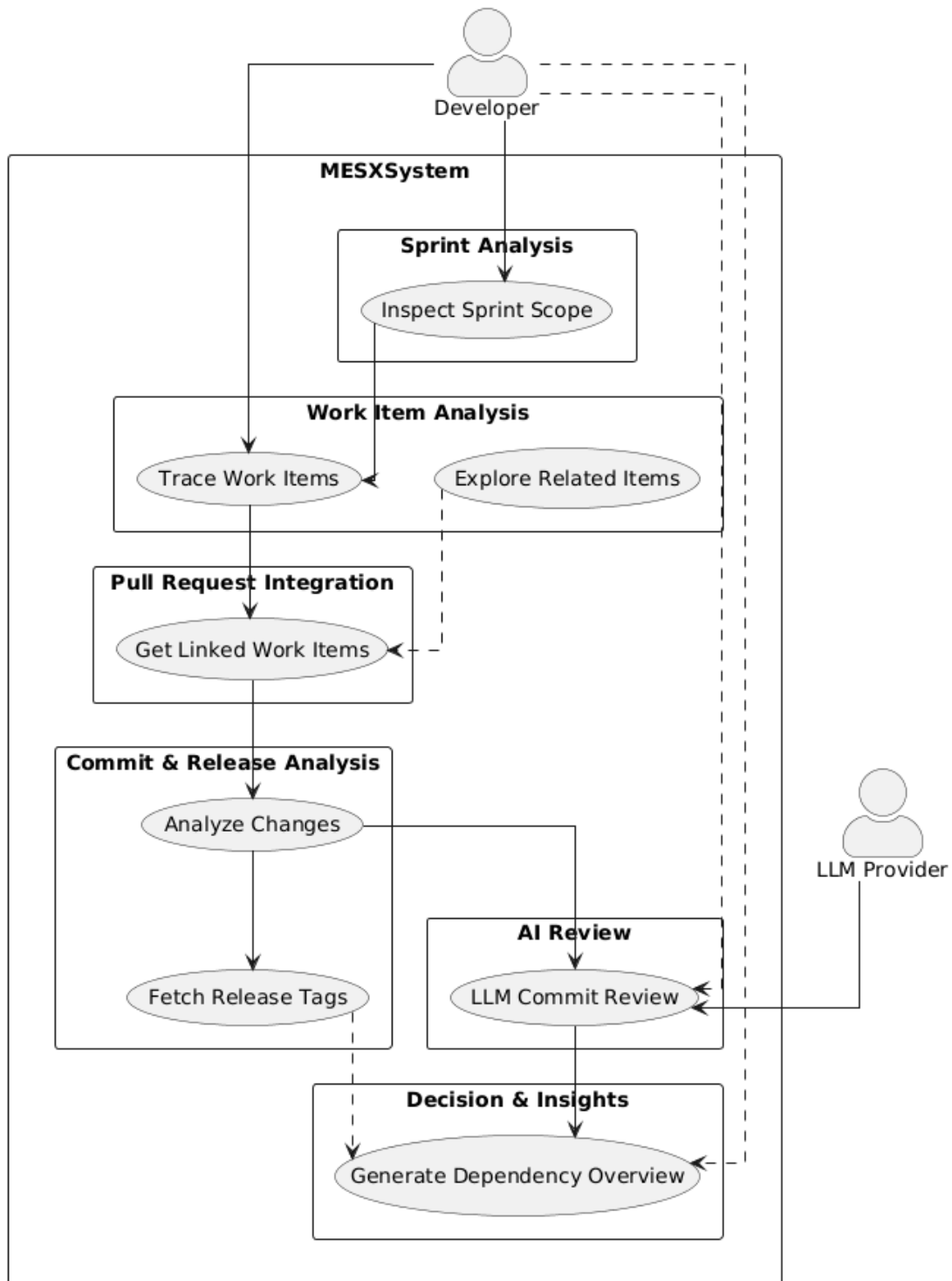


Figure 3.1. Use Case Diagram of the MES-X Dependency Management System.

3.9.4 Scope Boundaries

Two boundaries are worth stating clearly.

First, as set out in **FR-AN-05**, all use cases are read-only. The system does not create, modify, or delete anything in Azure DevOps. Azure DevOps appears in the model only as a data source.

Second, notification and alerting features, mentioned in some internal notes as possible future additions, are out of scope here. This specification covers only the traceability and analysis capabilities described above.

3.10 Requirements Traceability Summary

Table 3.1 shows how each functional requirement group maps to a system component. This makes clear which part of the system is responsible for which requirements, and creates the link between specification and design.

Table 3.1. Functional Requirements to System Component Traceability Matrix

Requirement Group	Requirement IDs	Responsible Component
Work Item Traceability	FR-WI-01 to FR-WI-06	Work Item Management Component
Pull Request Traceability	FR-PR-01 to FR-PR-04	Pull Request Resolution Component
Commit Traceability	FR-CO-01 to FR-CO-06	Commit Aggregation Component
Data Aggregation	FR-AG-01 to FR-AG-05	Decision Aggregation Component
Analysis and Reporting	FR-AN-01 to FR-AN-05	All components, Trace UI

3.11 Conclusion

This chapter set out the full requirements for the MES-X Dependency Management system. The functional requirements cover five areas: work item traceability, pull

request resolution, commit enrichment, data aggregation, and analytical reporting. The non-functional requirements set the bar for performance, scalability, reliability, maintainability, and security. The use case model translates those requirements into eleven concrete actor-system interactions.

Together, these requirements form the reference baseline for the rest of the report. They define not just what the system must do, but the level of performance, reliability, and maintainability expected of it in real use.

The next chapter presents the proposed architecture and explains how the chosen design addresses these requirements through a layered structure and a specialized traceability pipeline.

Chapter 4

Proposed Solution

4.1 Introduction

As discussed in the previous chapter, Azure DevOps holds all the data a team needs to trace a feature from planning through to deployment. The problem is not that the data is missing; it is that it is scattered. Work items, pull requests, commits, branches, tags, and sprint iterations each live in their own corner of the platform, with no built-in view that ties them together into a single coherent picture.

In practice, this means engineers and project managers must manually jump between multiple artifacts every time they want to understand the full lifecycle of a change. On a small project, that is manageable. In a microservices environment with dozens of repositories and hundreds of work items per sprint, it quickly becomes both time-consuming and unreliable.

This project addresses that gap directly. Instead of replacing Azure DevOps, we built a **centralized dependency traceability system** on top of it: a layer that collects data from Azure DevOps APIs, rebuilds relationships between artifacts, and exposes the result through one dedicated interface.

In practice, the system:

- fetches data from Azure DevOps APIs,
- rebuilds the dependency chain between artifacts,
- adds contextual metadata to raw data (branch position, release tags, repository status),
- supports sprint-driven work item selection as a natural entry point,

- offers optional AI-assisted commit review for deeper code-level insight,
- and displays results in a structured, actionable interface.

Three objectives guided the design:

- **Completeness.** The full dependency chain, from planning artifacts to code changes, should be visible in one place.
- **Consistency.** Data from different APIs and repositories must be standardized before processing, so results stay predictable even when teams configure projects differently.
- **Actionability.** The system should not only display data; it should help users answer concrete questions, such as whether a feature is ready for release or whether specific repositories still carry risk.

The rest of this chapter presents the architecture, the key design decisions, the core components, and the end-to-end flow that turns raw Azure DevOps data into structured traceability output.

4.2 Global System Architecture

The system is organized into three layers, each with a clear responsibility: the user interface, backend processing logic, and external integrations. Keeping these layers separate means that a change in one area, such as a breaking update in an Azure DevOps API, does not ripple through the rest of the system.

4.2.1 Presentation Layer: Trace UI

The Trace UI is the web interface through which users interact with the system. Its main role is to let users trigger dependency analysis, monitor progress in real time, and explore results in a structured way.

Beyond visualization, the Trace UI handles user input, calls the backend, and organizes returned data into analysis-friendly views. It supports four modes:

- single work item analysis,
- batch analysis of multiple items,

- sprint-driven selection, where the user picks a sprint and the system retrieves the associated work items automatically,
- and batch-related analysis, which combines dependency graphs across several items at once.

Each mode targets a different use case, from a quick spot-check on one feature to a full sprint-level analysis before a release.

4.2.2 Application Layer: Backend Services

The backend is where processing happens. It sits between the Trace UI and Azure DevOps, handling everything that requires business logic:

- constructing the dependency graph from work item relationships,
- normalizing data coming from different APIs into a consistent internal format,
- enriching artifacts with contextual information (branch position, tag data, repository status),
- retrieving work items associated with a sprint,
- coordinating optional LLM-assisted commit review,
- and computing the final release-readiness decisions.

Internally, the backend is organized into domain modules (Work Item, Pull Request, Commit, Sprint, Decision, and LLM), each responsible for one slice of the pipeline.

4.2.3 Integration Layer: Azure DevOps APIs

The integration layer handles all communication with Azure DevOps. It relies on two APIs:

- the **Work Item API**, used to retrieve work items, their relationships, and sprint iteration data,
- and the **Git API**, used to fetch pull requests, commits, branches, and tags.

When optional AI-assisted review is requested, the system also calls the Gemini API through a dedicated backend module.

All API calls go through shared utilities for authentication, request formatting, and error management. This keeps the rest of the system insulated from service-specific details.

4.2.4 Layer Interaction

Communication between layers follows a strict one-way flow:

UI → Backend → Azure DevOps

The Trace UI never calls Azure DevOps directly. All requests go through the backend first. This makes the system easier to test, easier to secure, and easier to maintain as APIs evolve.

4.2.5 Connection Graph

Figure 4.1 shows the full connection graph: which UI services invoke which backend modules, which modules access which Azure DevOps resources, and where the optional LLM path branches off.

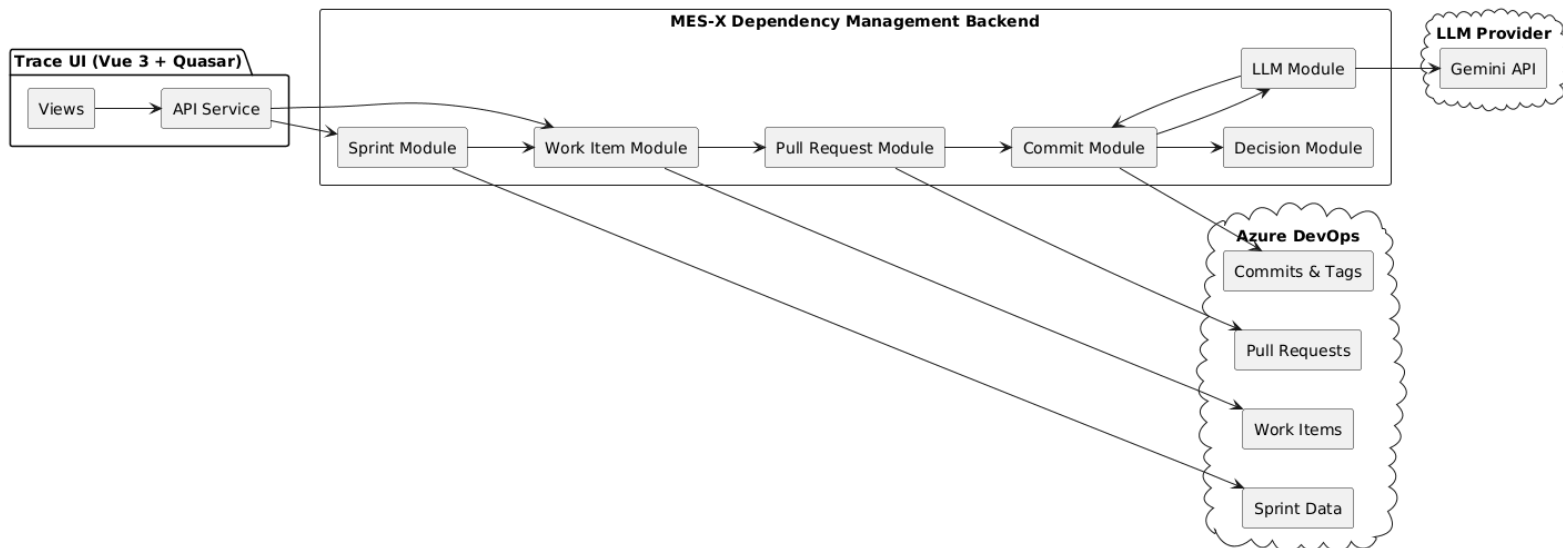


Figure 4.1. Global connection graph of the Trace UI and backend modules

4.3 Design Principles

Five principles guided the design of this system. Each one responds to a practical challenge identified during analysis.

4.3.1 Modularity

The system is divided into independent modules, each responsible for a well-defined task. Changes to one module, such as updating how commits are resolved, do not require touching others like sprint retrieval or decision aggregation. This keeps the codebase readable and reduces unintended side effects when the system evolves.

4.3.2 Separation of Concerns

Modules communicate only through well-defined data transfer objects (DTOs). No module reaches into another's internal logic. This boundary is what makes the system predictable: each module can be understood and tested in isolation, and the interfaces between them are explicit.

4.3.3 Data Normalization

Azure DevOps responses are not always consistent across projects, teams, or API versions. Before processing, data is converted into a stable internal format. This normalization layer acts as a buffer between raw API output and the rest of the system, so business logic does not need to account for external inconsistencies.

4.3.4 Batch Processing

Rather than fetching items one by one, the system groups requests wherever possible. This reduces API traffic, speeds up analysis, and helps avoid Azure DevOps rate limits. For large projects with hundreds of work items and dozens of repositories, this is the difference between a responsive tool and an unusable one.

4.3.5 Assisted Decision Support

A clear line separates deterministic traceability logic from optional AI assistance. Release-readiness decisions are always computed from Azure DevOps data using fixed

rules and never depend on the LLM. The LLM path exists only to provide additional review insight on selected commits, as a complement to core logic.

4.4 Core Components

The backend is organized into six components that form a linear processing pipeline:

Sprint Selection → Work Items → Pull Requests → Commits → Decisions (+
Optional LLM Review)

Each stage adds a layer of context to the data before passing it to the next.

4.4.1 Sprint Exploration Component

This component provides a planning-oriented entry point into the traceability pipeline. It loads available sprints for a project and retrieves the work items associated with the selected sprint. The resulting work item IDs are then handed off to the same pipeline used for manual selection, so the rest of the flow is identical regardless of how analysis started.

4.4.2 Work Item Management Component

This is where the dependency graph takes shape. Starting from one or more work items, the component explores all relationships (parent/child hierarchies, related items, and pull request links) and expands them recursively until the full dependency tree is collected.

To keep this process efficient, it relies on graph traversal with batching and caching, so the same item is never fetched twice and API calls are grouped wherever possible. The output is a complete list of related work items, along with associated pull requests.

4.4.3 Pull Request Resolution Component

Pull requests are the bridge between the planning world (work items, sprints) and the code world (commits, branches). This component fetches details for each pull request identified in the previous step and maps them to their repositories.

Without this step, it would be impossible to know which code changes correspond to which features.

4.4.4 Commit Aggregation Component

This component retrieves the actual code changes. It collects commits from pull requests and direct links, removes duplicates across repositories, and augments each commit with branch and tag information so each change can be positioned in the release timeline.

It also exposes the review entry point used by the decision view: when a user wants LLM feedback on a specific commit, this component handles the handoff to the LLM module.

4.4.5 Decision Aggregation Component

Rather than fetching new data, this component analyzes everything collected in previous stages and produces a structured release-readiness assessment. It answers questions such as:

- Are all work items in the dependency graph completed?
- Are the relevant commits included in the current release branch or tag?
- Which repositories are in a ready state, and which ones have outstanding issues?

The output is a clear, per-repository decision view that teams can use directly during release evaluation.

4.4.6 LLM-Assisted Review Component

This component is entirely optional and only runs when explicitly requested. Given a specific commit, it builds a review prompt using commit metadata, changed files, and any configured repository-level guidance. It then calls the external model and returns the review payload to the UI.

This component does not influence deterministic decisions produced by the previous stage. It exists as a separate, on-demand tool for teams that want an additional layer of code-level analysis.

4.5 End-to-End Data Flow

From the moment a user starts an analysis to the moment results appear on screen, the system follows a consistent sequence:

1. The user initiates an analysis from the Trace UI, either by selecting work items manually or by choosing a sprint.
2. If sprint mode is used, the system fetches the sprint's work items and forwards their IDs into the pipeline.
3. The Work Item component retrieves each item and expands the full dependency graph.
4. The Pull Request component resolves the pull requests linked to those work items.
5. The Commit component collects all commits, deduplicates them, and adds branch and tag context.
6. The Decision component aggregates everything into a release-readiness view.
7. Optionally, the user can trigger an LLM review for specific commits from within the decision view.
8. All results are displayed in the Trace UI's trace and decision views.

At each step, the data becomes richer and more meaningful. Raw work item IDs become a dependency graph; commits are placed in a release timeline; individual statuses become an aggregated readiness assessment.

4.6 User Interface and Interaction

The Trace UI is designed around the four usage modes described earlier: single item analysis, batch analysis, sprint exploration, and batch-related analysis. Each mode is available from the same interface and feeds the same backend pipeline.

One deliberate design choice was to use streaming for real-time feedback. Instead of showing a single loading state while the backend completes the full analysis, the UI reports progress as each stage finishes. For large analyses, this noticeably improves the user experience.

From the decision view, users can also trigger an LLM commit review directly, without leaving the interface. This keeps the workflow in one place instead of requiring users to copy commit hashes into another tool.

4.7 Observability and Logging

The backend uses a custom `AppLogger` class for structured logging throughout the system. This logger extends the shared logging service used in the project and is initialized through the logger configuration module so that the minimum log level can be loaded from the environment or from the application configuration store.

This makes it easier to understand what happened during an analysis and to diagnose problems when they occur. Four log levels are used:

- `log` for normal operations,
- `warn` for minor issues that do not break the analysis but are worth noting,
- `error` for failures that affect results,
- and `debug` for detailed internal state, useful during development.

Taken together, the components and design decisions described in this chapter form a system that transforms scattered Azure DevOps data into a structured, end-to-end traceability view. The combination of deterministic dependency analysis, sprint-oriented entry points, and optional AI-assisted review gives teams the visibility they need to make informed release decisions, without having to piece the picture together manually each time.

Chapter 5

Implementation

5.1 Introduction

This chapter covers the technical implementation of the MES-X dependency traceability system, describing how the proposed architecture was built into a working full-stack application. The system brings together work items, pull requests, commits, branches, tags, sprint planning data, and AI-assisted review into a single coherent workflow.

The chapter starts with the technology stack, then walks through the development approach, backend modules, frontend integration, deployment setup, and the quality practices used to validate the final product.

5.2 Technology Stack and Architecture

MES-X is built on a modern full-stack architecture designed to handle large-scale dependency graph reconstruction and traceability analysis efficiently.

5.2.1 Web Application Layer (Frontend)

The frontend is the main interface operators use to run and explore traceability analyses. It is built for responsiveness and clarity when rendering complex dependency graphs.

Technical specifications:

- **Framework:** Vue 3 with TypeScript
- **UI framework:** Quasar component system

- **Build tool:** Vite
- **Purpose:** trace execution, progressive visualization, and dependency navigation



Figure 5.1. Frontend technology used: Vue 3 for building the user interface

5.2.2 Backend Service Layer

The backend is the core of the system. It handles traceability logic and exposes structured APIs for both single and batch analysis.

Technical specifications:

- **Framework:** NestJS (TypeScript)
- **API styles:** REST for queries, SSE for streaming
- **Role:** orchestrating dependency graph construction

The backend is organized into domain-driven services, each responsible for processing and enriching a specific part of the traceability data.



Figure 5.2. Backend technology used: NestJS for scalable server-side architecture

5.2.3 Domain Service Layer

This layer holds the core business logic of the system:

- Work item graph traversal and relationship resolution
- Pull request linkage and analysis
- Commit extraction and repository context resolution
- Sprint-to-work-item retrieval for seeding batch traces
- LLM-assisted commit review with project-specific guidance
- Tag and branch contextual enrichment

Key responsibilities:

- dependency graph construction
- deduplication and cycle prevention
- structured data enrichment for UI consumption

5.2.4 Cloud Infrastructure and DevOps

This layer handles deployment, integration, and operational automation.

- **Source control:** Azure DevOps Git repositories
- **CI/CD pipelines:** automated build, test, and deployment workflows
- **Containerization:** Docker for consistent environment execution
- **External APIs:** Azure DevOps Work Item and Git APIs

This setup ensures the system is reproducible, scalable, and deployable across environments with minimal configuration overhead.



Figure 5.3. Azure DevOps Platform

5.3 Development Methodology

5.3.1 Agile Scrum Execution Model

Development followed an Agile Scrum approach suited to the incremental nature of building a traceability system.

- **Sprint structure:** short iterations focused on incremental feature delivery
- **Backlog management:** user stories prioritized by traceability value and complexity
- **Rituals:** sprint planning, daily stand-ups, reviews, and retrospectives
- **Tracking tool:** Azure DevOps Boards



Figure 5.4. Agile Scrum methodology applied during system development

5.3.2 Implementation Phases

MES-X was built module by module using a continuous integration strategy. Each module went through the full software lifecycle: design, implementation, testing, integration, and deployment.

This approach allowed early validation of core components and kept integration risks low by continuously aligning the backend, frontend, LLM features, and DevOps pipelines.

Development was split into seven phases: Setup, Work Items, Pull Requests, Commits, Sprint Exploration, LLM-assisted Review, and Final Release.

Phase 1: Setup and Infrastructure Foundation

The first phase prepared the development environment and core delivery pipeline.

Main objectives:

- developer onboarding and environment preparation;
- local infrastructure setup and database connections;
- CI/CD pipeline initialization;
- frontend project structure and build pipeline creation.

Key outcomes:

- fully operational development environment;
- automated build pipeline ready for integration;
- initial frontend architecture and design system foundation.

Phase 2: Work Items Module

This phase introduced the first core functional module: work item management and traceability.

Backend implementation:

- work item data model design and implementation;
- service layer logic;

- REST API endpoints for querying and filtering;
- validation and schema enforcement.

Frontend implementation:

- work item listing and filtering interface;
- creation and editing forms;
- detailed view with status tracking.

Quality and delivery:

- backend service unit tests;
- end-to-end UI workflow tests;
- staging deployment and production release.

Phase 3: Pull Requests Module

This module connected development activity to work items through pull requests, enabling traceability between code changes and planning items.

Backend implementation:

- pull request data model design;
- storage and filtering logic;
- API endpoints for PR retrieval and aggregation;
- statistical enrichment of PR data.

Frontend implementation:

- PR dashboard and listing view;
- diff viewer for code comparison;
- review and approval workflow interface;
- status indicators and activity timeline.

Delivery process:

- API and data model documentation;
- unit and integration testing;
- staging validation and production rollout.

Phase 4: Commits and Graph Traceability Engine

This phase is the heart of the MES-X system. It introduced deep graph traceability across commits, branches, and repository structure.

Backend implementation:

- commit data model and storage layer;
- graph traversal algorithms for dependency exploration;
- cycle detection to prevent infinite traversal;
- performance tuning for large repositories.

Frontend implementation:

- commit history and pagination view;
- interactive graph visualization of dependencies;
- commit details and file-level diff inspection.

Validation and testing:

- unit tests for traversal and API logic;
- UI-based end-to-end graph testing;
- performance validation against large datasets.

Phase 5: Sprint Module and Planning Integration

Once the core traceability graph was stable, the system was extended with a Sprint module to bridge planning data and dependency analysis.

Backend implementation:

- sprint listing endpoint backed by Azure DevOps iterations;

- sprint work item retrieval through iteration relations;
- DTO contracts for sprint metadata and work item payloads;
- controller and service-level logging and error handling.

Frontend implementation:

- dedicated sprint exploration screen;
- lazy loading of work items when a sprint is expanded;
- cross-sprint work item selection before launching batch analysis.

Validation and delivery:

- controller and service unit tests for sprint queries;
- manual validation of sprint loading and selection flows;
- integration of sprint-selected work items with the batch traceability workflow.

Phase 6: LLM-assisted Review

This phase added an LLM-assisted review feature to support commit inspection directly from the traceability workflow.

Backend implementation:

- a dedicated LLM module exported to the Commit module;
- prompt construction from commit metadata, changed files, and code snapshots;
- injection of repository-specific guidance from instruction and skill files;
- result caching and retry handling for external model rate limiting.

Frontend implementation:

- LLM review action exposed from the decision dashboard at commit level;
- review dialog for displaying generated findings to the operator;
- direct coupling between decision results and commit review requests.

Quality and safeguards:

- human validation remained mandatory before acting on generated reviews;
- prompt scope was limited to relevant files and truncated snapshots;
- LLM failures were isolated from the core traceability workflow.

Phase 7: System Release and Final Integration

The final phase stabilized the system and prepared it for full production deployment.

Activities included:

- integration of all modules into a unified system;
- automated CI/CD validation stages;
- production deployment pipeline configuration;
- post-deployment monitoring and validation.

Final outcome:

- fully integrated MES-X traceability platform;
- stable, production-ready deployment pipeline;
- validated end-to-end dependency traceability workflow.

5.3.3 Module-Level Architecture

The backend runtime is organized into domain-focused modules:

- **Work Item module:** endpoints for single-root and batch traversal, and SSE task streaming.
- **Pull Request module:** linked work item retrieval by PR and PR summary enrichment.
- **Commit module:** commit lookup by work item and PR, branch/tag context retrieval, batch tag resolution, and commit review orchestration.
- **Sprint module:** sprint listing by project and sprint work item retrieval for seeding batch analysis.

- **Decision module:** aggregation service that synthesizes traceability outputs for decision support.
- **LLM module:** commit review prompt generation, guidance collection from repository instructions, external model invocation, and response caching.

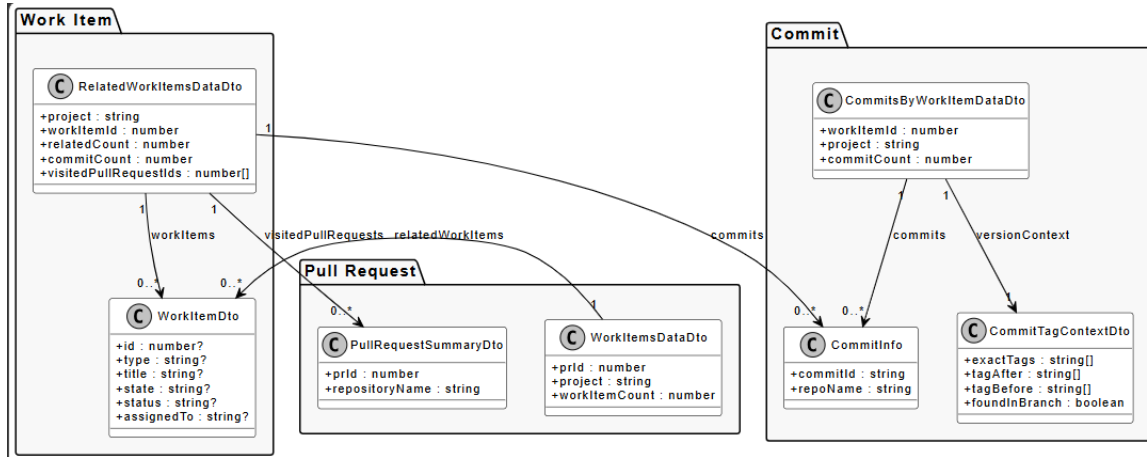


Figure 5.5. Backend class diagram for the work item, pull request, and commit modules

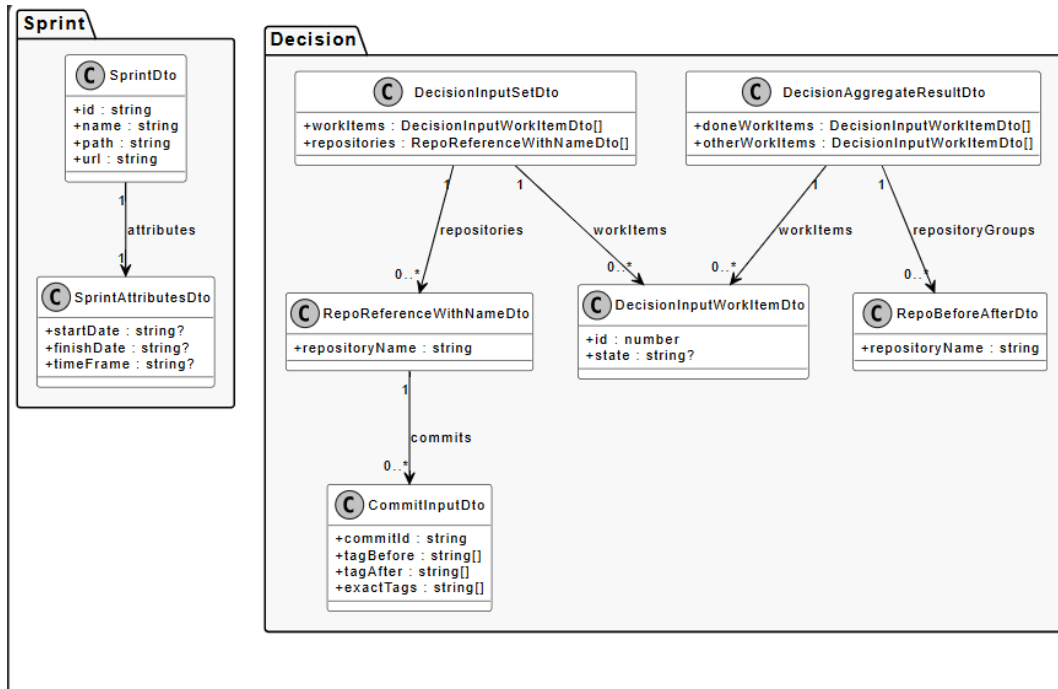


Figure 5.6. Backend class diagram for the sprint and decision modules

5.3.4 API Contract Design

The backend exposes a set of RESTful endpoints that support both single and batch traceability analysis with a consistent response model.

Single related endpoint `/workitem/:id/related`

Returns a traceability graph centered on a single root work item.

Batch related endpoint `/workitem/related`

Returns an aggregated, deduplicated graph for multiple root items.

Tasks stream endpoint `/workitem/tasks/stream`

Streams processing results progressively using Server-Sent Events (SSE).

PR-to-work-items endpoint `/pullrequests/:prId/workitems`

Retrieves work items linked to a given pull request.

Sprint listing endpoint `/sprint?project=...`

Returns the iterations available for a project.

Sprint work-items endpoint `/sprint/:sprintId/work-items?project=...`

Returns work items for a given sprint and supports lazy loading in the UI.

Commit tag batch endpoint `/commits/tags/batch`

Enriches commit references with repository tag metadata.

LLM commit review endpoint `/commits/hf/review`

Generates an AI-assisted review for a selected commit using repository and commit context.

The DTO strategy prioritizes:

- stable response contracts between the backend and its consumers;
- best-effort metadata enrichment (e.g., repository names for visited PRs);
- a clear separation between operational traceability contracts and optional AI-generated review payloads;
- explicit counters and trace metadata for UI sorting and interpretation.

5.3.5 Implementation Characteristics

- graph traversal with visited-state tracking to prevent cycles;
- request-scoped caching and batched retrieval to reduce external API load;
- normalization of external payloads before propagation between services;
- structured logging to support diagnostics and operational monitoring.

5.3.6 Sequence and Processing Flows

Backend processing is built around orchestration pipelines where each service builds on the output of the previous stage.

- **Single-root flow:** resolve the root work item, traverse related nodes, enrich pull requests and commits, return a normalized graph.
- **Batch flow:** run multi-root traversal with deduplication and consolidated counters.
- **Sprint exploration flow:** load project sprints, lazily fetch sprint work items, then forward selected items to the batch flow.
- **Commit tag flow:** resolve tag context in batch, grouped by repository, using a latest-tag strategy.
- **LLM review flow:** collect commit metadata and relevant file snapshots, augment the prompt with repository guidance, invoke the model, and return the generated review to the decision UI.

Single-Root Related Work Item Processing

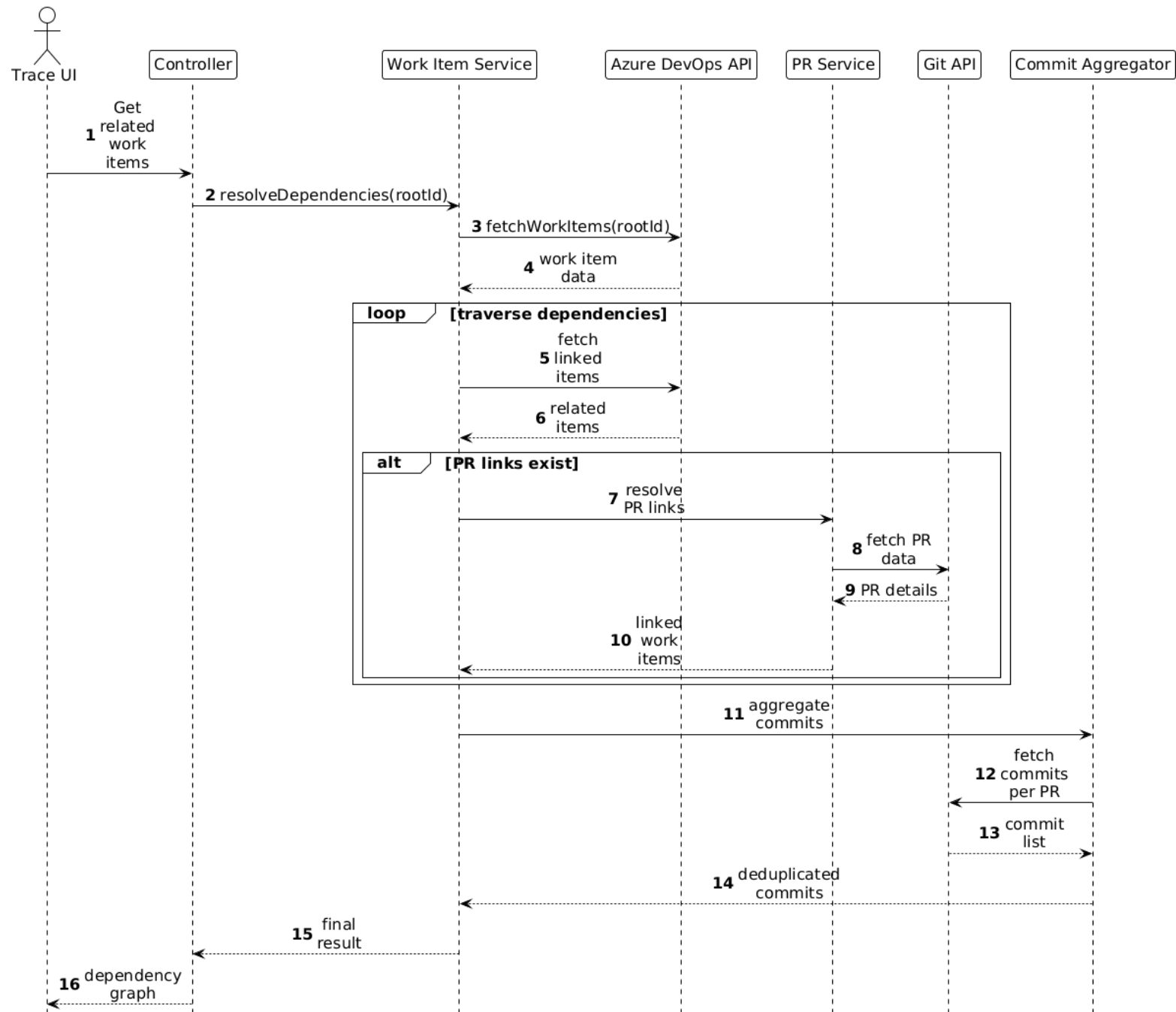


Figure 5.7. Sequence diagram for single-root related work-item processing

Batch Related Work Item Aggregation

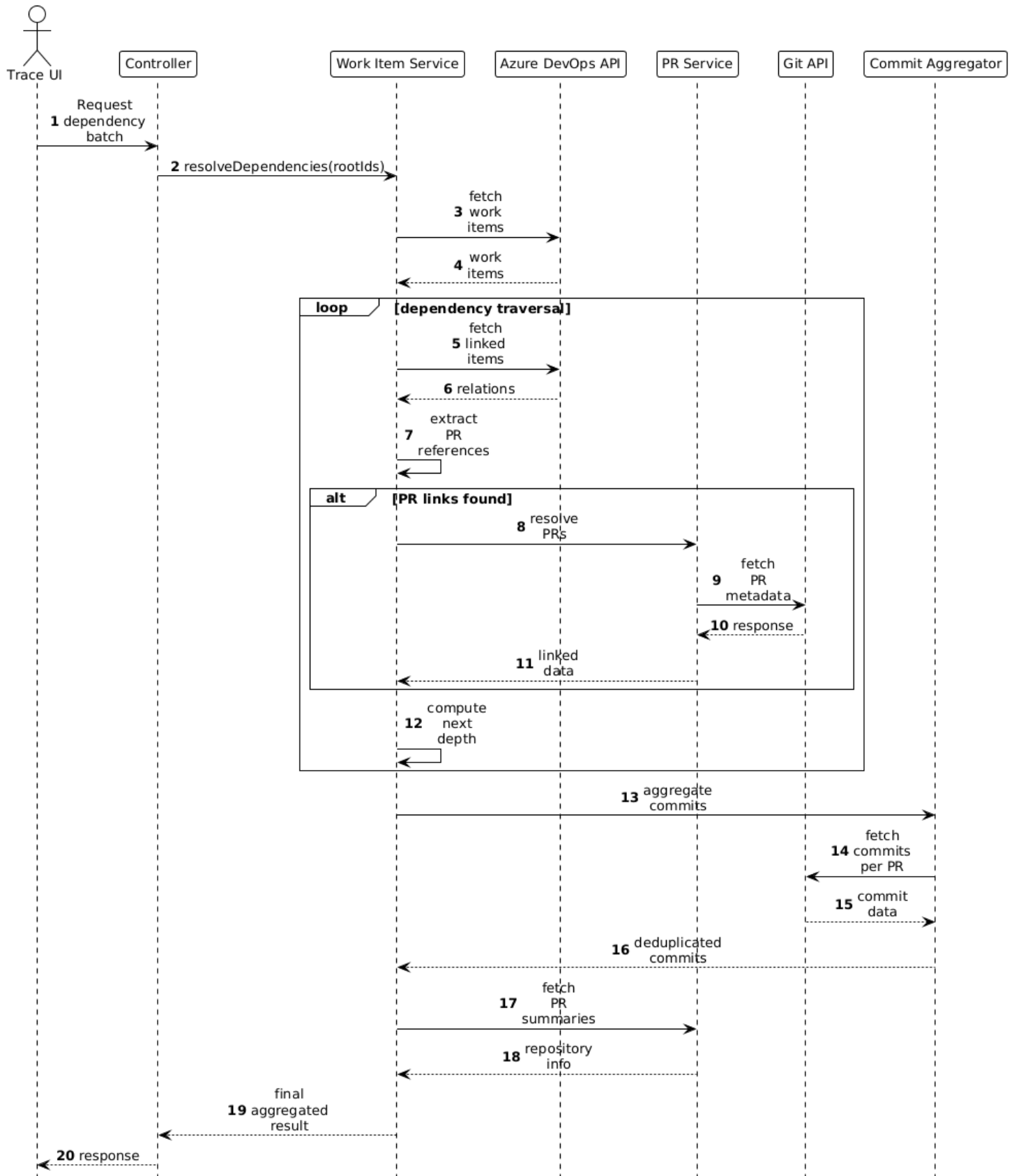


Figure 5.8. Sequence diagram for batch related work-item aggregation

5.4 UI Implementation

5.4.1 UI Architecture and Main Surfaces

The UI layer is the main entry point to the traceability system. It is organized around route-driven views that map to the key analysis scenarios, with a shared state and API integration layer handling requests, progressive loading, and result display.

- **RelatedWorkItemTrace:** single-root analysis screen for exploring one work item and its related artifacts.
- **BatchWorkItemTrace:** selection and loading screen for progressively collecting candidate work items through SSE streaming.
- **SprintWorkItemsView:** planning-oriented screen for loading sprints, lazily fetching their work items, and preparing cross-sprint selections.
- **BatchRelatedWorkItemTrace:** aggregated result screen showing the consolidated output of multi-root analysis with commit and repository context.
- **DecisionDashboard:** decision and promotion screen extended with commit-level LLM review actions.

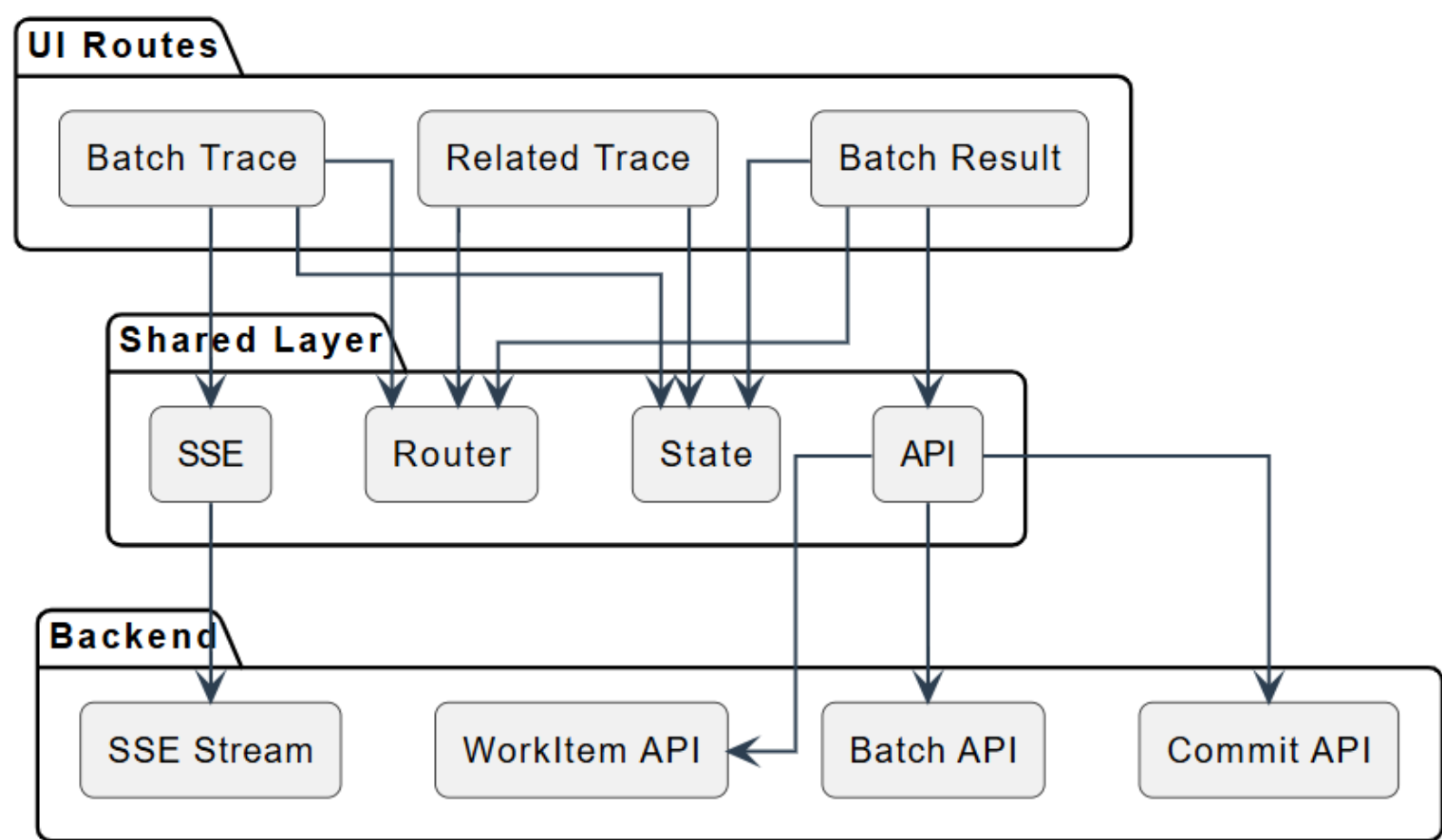


Figure 5.9. UI route and component architecture for traceability workflows

5.4.2 UI Data Flow and Backend Integration

The UI communicates with the backend through a dedicated trace API layer and maps backend contracts to typed frontend models.

- Single analysis uses `/workitem/:id/related`.
- Batch task loading consumes `/workitem/tasks/stream` via SSE.
- Sprint exploration calls `/sprint` and `/sprint/:sprintId/work-items` with lazy retrieval per expanded sprint.
- Batch result enrichment uses `/commits/tags/batch` grouped by repository.
- Commit-level review actions call `/commits/review` from the decision dashboard when the operator requests an AI-generated assessment.
- Authentication failures trigger a controlled redirect when the backend returns an unauthorized response.

Traceability UI - Backend Integration

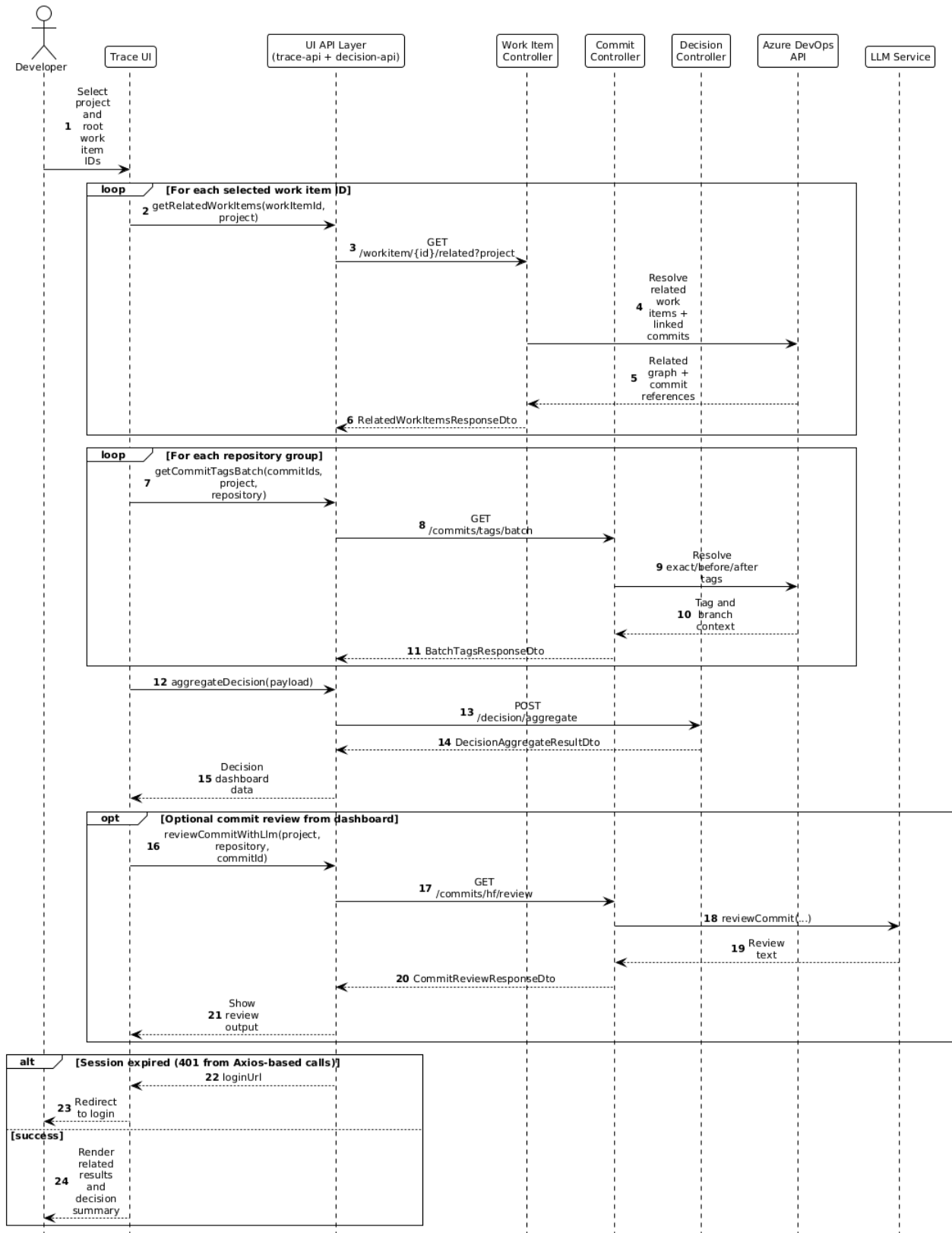


Figure 5.10. Sequence diagram for UI and backend integration in trace workflows

5.4.3 UI Operational Considerations

- progressive loading to keep batch interactions responsive;
- consistent result ordering based on traceability counters;
- graceful rendering when enrichment metadata is partially unavailable.

5.4.4 Screens and UI Walkthrough

Batching Work Items

This screen is used to select multiple work items for batch processing and kick off a batched traceability run. Selection is flexible — single items, multiple, or all at once — and run progress is shown progressively using Server-Sent Events (SSE). The screen includes quick filters and tools for prioritizing candidates before downstream aggregation.

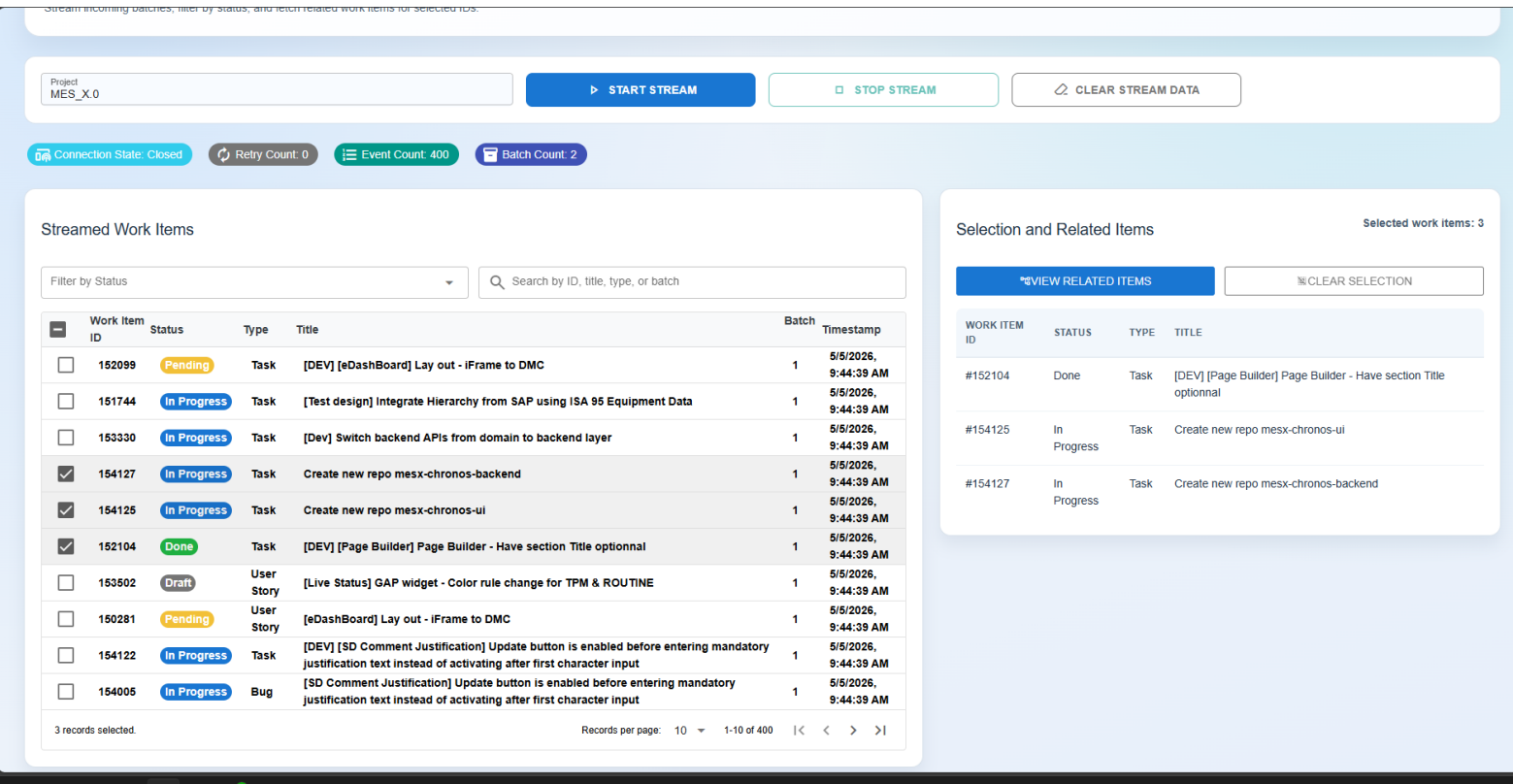


Figure 5.11. Batching Work Items – selection and SSE-driven progress

Get Related

The "Get Related" flow has two complementary screens:

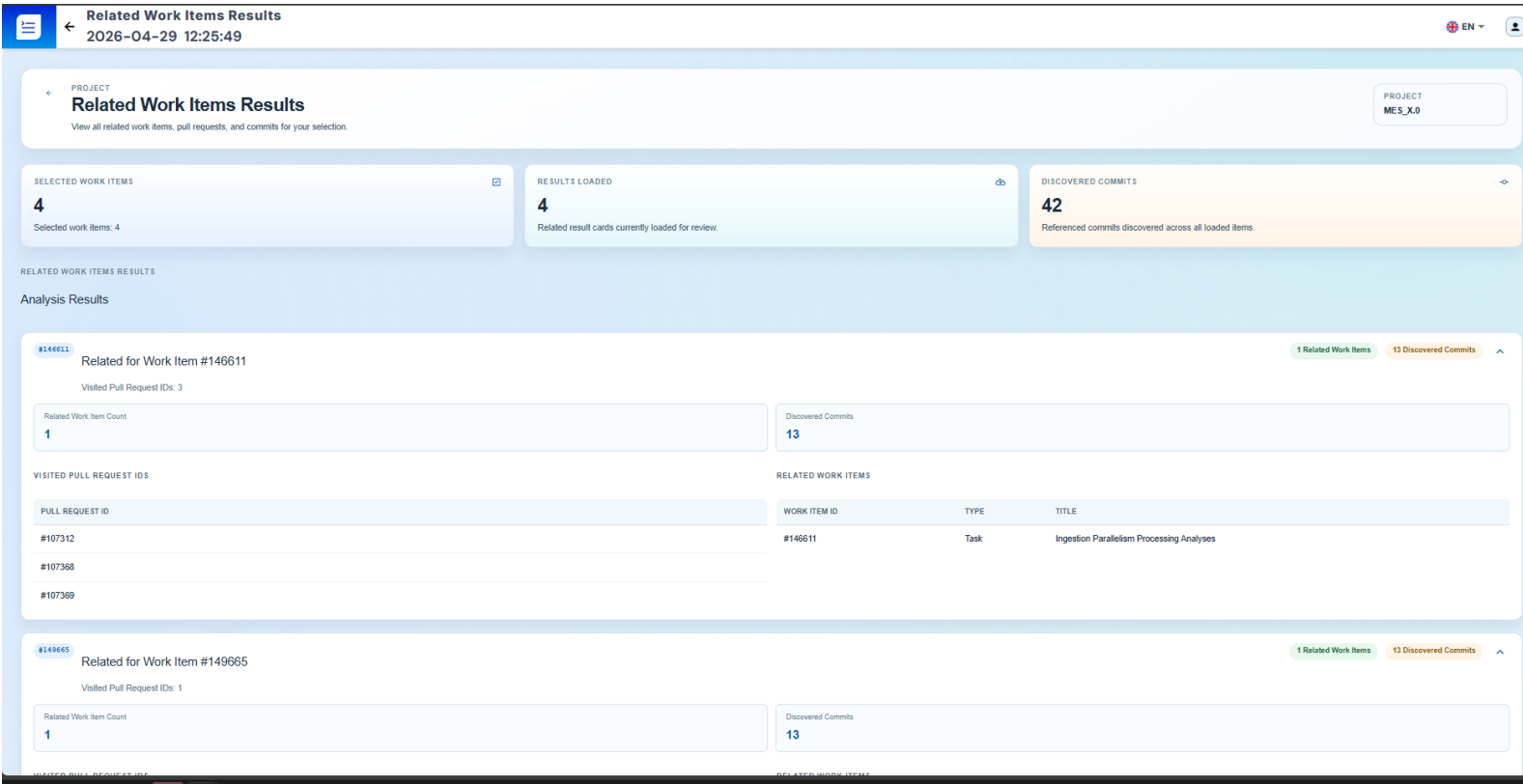


Figure 5.12. Related Work Item – single-root traceability graph view

The Related Work Item view shows a single root work item and its connected graph (pull requests, commits, branches, tags). Nodes can be expanded and inspected, and the operator can drill down to commit or PR level.

The Discovered Commits screen lists commits found during traversal along with contextual metadata: branch, tags, and associated work items. This view is useful for verifying commit-to-work-item linkage and checking tag context before any automated decisions are made.

Sprint Work Items

The Sprint Work Items screen provides a planning-driven entry point to the traceability workflow. Rather than starting from manually entered identifiers, the operator loads the sprint list for a project and expands individual sprint cards to lazily fetch their work items. Selected items are kept across loaded sprints and can be forwarded directly to the batch analysis flow.

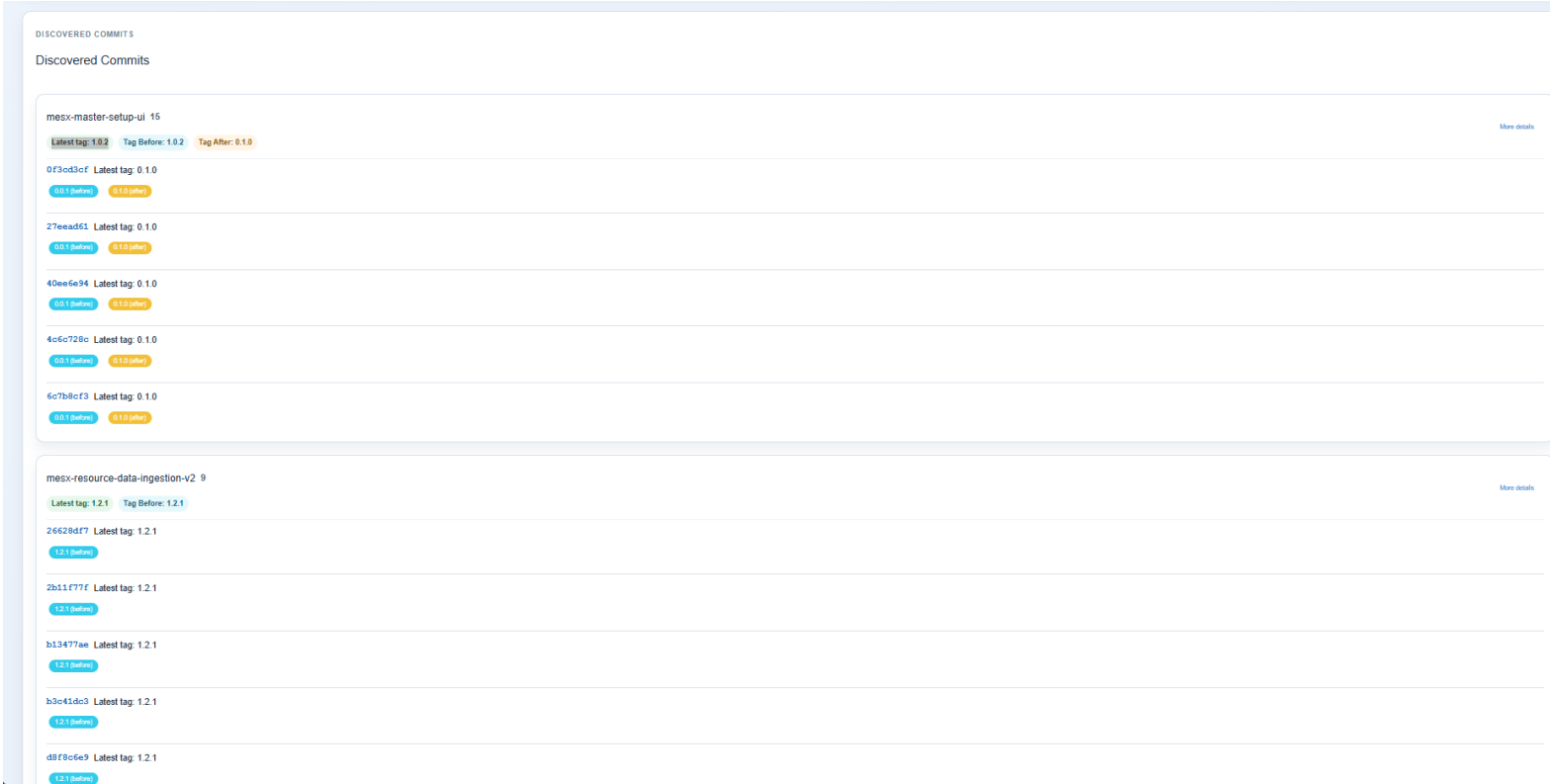


Figure 5.13. Discovered Commits – commits found during traversal

Decision

The Decision section provides summarized outputs to guide promotion actions. It contains two key screens:

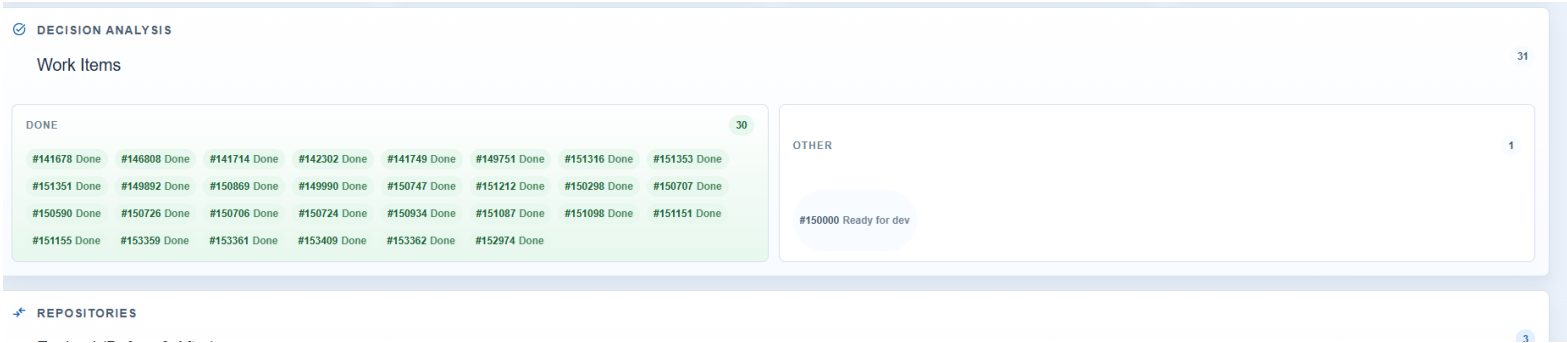


Figure 5.14. Decision – work items grouped by state (Done vs. Other)

The first Decision view groups work items by state (for example, Done vs Other) so operators can quickly see which items are ready for promotion and which still need attention.

The repository/tags screen shows, per repository, the commits and expected tags (before/after/exact) that the system identified. From here, an operator can select a candidate work item and trigger a promotion workflow (for example: move a work

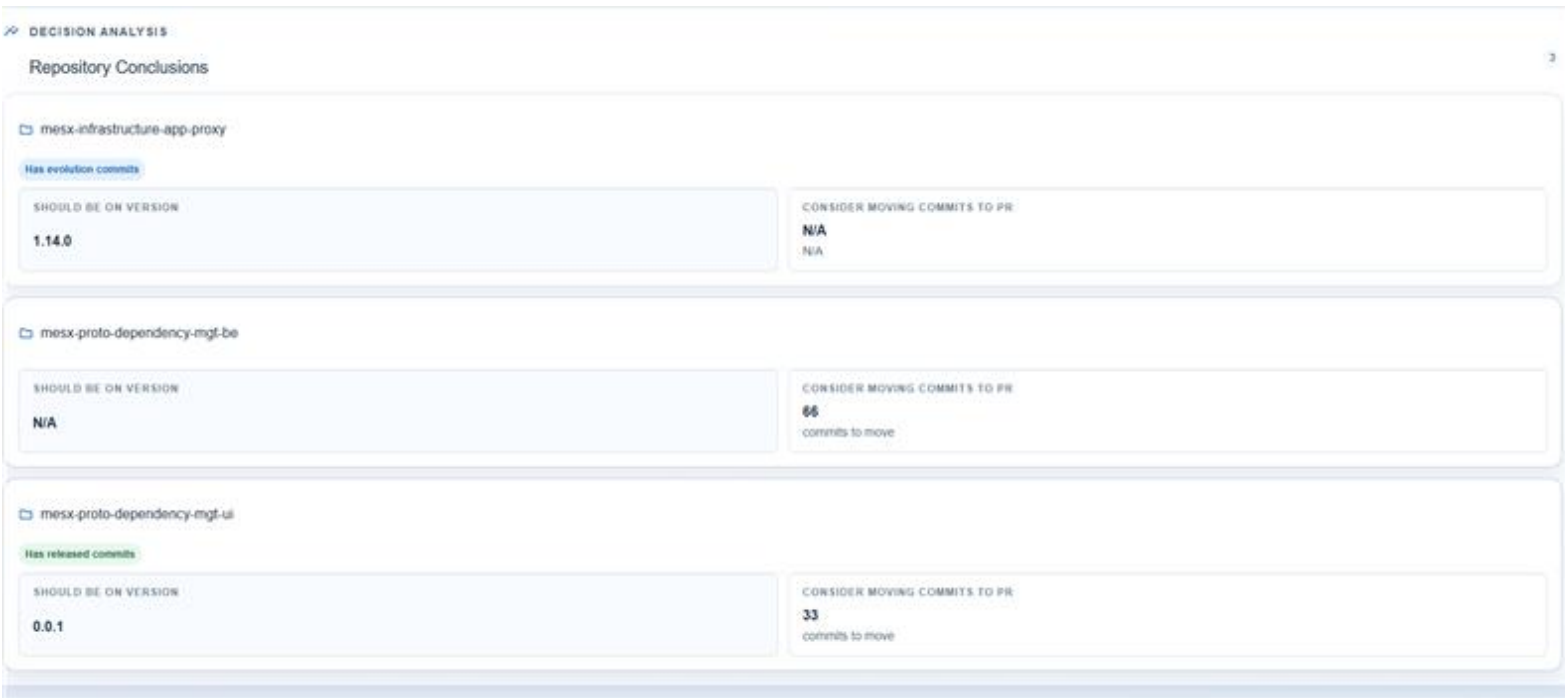


Figure 5.15. Decision – repository view with expected tags for promotion

item to production) after reviewing tag context. The same screen also exposes an LLM review action on eligible commits, allowing the operator to request an AI-generated assessment before making a final decision.

5.5 Docker Orchestration and Deployment

5.5.1 Service Orchestration Model

The backend runtime is packaged as an isolated service with clearly defined integration boundaries.

- The backend container exposes REST/SSE APIs on the configured runtime port.
- Runtime configuration and secrets are injected through environment variables and externalized config files.
- The backend accesses Azure DevOps Work Item and Git APIs over HTTPS.

5.5.2 Operational Dependencies

- CI/CD pipeline for image build and deployment validation.
- Optional caching layer through internal cache services.

- Secrets and environment values injected at runtime.

5.5.3 Deployment Objectives

- reproducible execution locally and in CI;
- independent versioning and rollout of backend artifacts;
- reduced deployment risk through clear component ownership.

5.6 Quality, Security, and Operational Considerations

5.6.1 Testing and Validation

Validation was carried out at multiple levels to ensure the delivered solution matched the requirements defined earlier in this report.

- Backend unit tests covered controller and service behavior, especially work item traversal, commit resolution, and DTO mapping.
- End-to-end tests validated the main API flows and confirmed the frontend correctly consumed the backend contracts.
- Frontend quality checks combined linting, style validation, and manual testing of the main traceability views and the decision dashboard.
- Cross-layer checks ensured that streaming behavior, repository grouping, and decision outputs remained consistent from backend to UI.

5.6.2 Developed Tests

The test effort focused on validating the most critical traceability services and API contracts used by the UI.

- **Backend unit tests (Jest):** implemented for the Work Item, Commit, and Sprint modules, including controller and service-level behavior.
- **Backend utility tests:** added for tag-selection logic used by repository context enrichment.

- **Backend API end-to-end test:** implemented under the `test/` suite to validate core HTTP integration behavior.
- **Frontend validation:** performed through linting, static checks, and manual scenario-based verification of key traceability screens.

Quality controls:

- controller-level input validation and typed DTO contracts;
- service-level tests for traversal, tag selection, and data normalization;
- regression-focused tests for traversal and normalization behavior.

Security and runtime controls:

- secure communication for all backend integrations;
- managed access to Azure DevOps external APIs;
- structured logging to support incident analysis and diagnostics.

5.7 AI-assisted Development and Review

AI-assisted tools were used throughout the project to support both backend and frontend work. The goal was not to replace engineering decisions, but to speed up repetitive tasks, improve review coverage, and encode project-specific conventions.

The application also ships an operational LLM capability within the backend itself. The Commit module delegates review requests to a dedicated LLM module that builds prompts from Azure DevOps commit metadata, selects a small set of relevant changed files, injects repository instructions and skill guidance, and calls an external model to produce a structured review. This review is then surfaced in the decision dashboard, so release analysis can be complemented by code-level observations when needed.

Both repositories keep their automation artifacts inside the codebase. On the backend, the `.github/agents/` directory defines dedicated roles such as architect, implementer, reviewer, and QA agent. On the frontend, `.github/agents/`, `.github/instructions/`, and `.github/skills/` capture UI guidelines, review rules, and reusable workflows for Vue, Quasar, styling, and quality checks. Keeping these files versioned alongside the source code makes the automation explicit, reviewable, and reproducible.

In practice, these agents assisted with code drafting, refactoring, test scaffolding, documentation, and architecture reviews. The instruction files helped enforce conventions such as typed contracts, logging discipline, component structure, localization, and style consistency. On the frontend they reinforced guardrails around Vue Composition API usage, token-based styling, and UI quality. On the backend they kept DTO usage, controller-service separation, and observability patterns consistent across the codebase.

All AI-generated suggestions went through human review. Proposed changes were treated as drafts and validated through linting, SonarQube policies, tests, and direct developer inspection before being merged. This made AI tools genuinely useful across both development and runtime review support, while keeping accountability, technical rigor, and architectural coherence firmly in human hands.

Conclusion

The implementation produced a modular traceability platform combining a graph-oriented backend, an operational frontend, and a deployment-ready runtime configuration. The result replaces a manual, fragmented analysis process with a structured workflow that is faster, easier to interpret, and better suited to release preparation.

By combining layered design, iterative development, thorough validation, and careful integration with Azure DevOps, the project built a maintainable foundation for dependency analysis, release decision support, and future expansion of the traceability scope.

Conclusion

Introduction

This project started from a simple real-world problem: release preparation was slow, manual, and spread across multiple Azure DevOps screens. The goal of MES-X was to make that process clearer and more reliable by bringing related artifacts into one traceability workflow. During the internship, the work covered the full cycle: architecture, implementation, integration, validation, and deployment preparation.

Key Achievements

The delivered system includes a modular backend and a practical frontend built around daily traceability needs.

On the backend, dedicated modules were implemented for Work Items, Pull Requests, Commits, Sprints, Decision aggregation, and LLM-assisted review. These modules expose clear REST contracts, plus SSE streaming for progressive task loading.

On the frontend, the main operator screens were implemented: single-root traceability, batch work item loading with live progress, sprint exploration, batch related results, and a decision dashboard. Together, these screens allow the user to go from selecting work items to reviewing dependency context and promotion-oriented summaries in one flow.

Technical Impact

Technically, the biggest impact was moving from isolated lookups to a coherent pipeline.

The backend handles recursive relationship traversal with cycle prevention, commit discovery, and repository tag context enrichment. DTO-based contracts made backend responses predictable and easier to consume. Batch-oriented operations reduced

repeated calls in the heavy parts of analysis.

On the UI side, typed service layers, explicit error handling, and fallback rendering improved robustness. SSE streaming improved usability by showing progress during long operations instead of leaving users waiting with no feedback.

Business Value

The business value is clear: less manual effort, better visibility, and faster release-readiness decisions.

By automatically linking work items, pull requests, commits, and tag context, the platform reduces investigation time and helps teams spot missing context earlier. Decision discussions are also easier because stakeholders can work from one consolidated view instead of navigating many disconnected pages.

Lessons Learned

This implementation highlighted several practical lessons.

First, incremental delivery worked well: building module by module made integration safer and easier to debug. Second, strict contracts between backend and frontend prevented many regressions. Third, observability matters: logging and clear error handling were essential for diagnosing multi-step flows.

For AI integration, the key lesson was to keep it assistive, not mandatory. The LLM review feature adds value, but the core traceability flow remains usable without it, and final decisions stay in human hands.

Future Perspectives

Future improvements are already clear from the current version.

Testing can be extended with broader end-to-end scenarios and stronger CI quality gates. Performance can improve with deeper caching and wider adoption of batch-oriented calls across UI flows. On the product side, the decision dashboard can be enriched with clearer trend views and explainability support.

Another promising direction is broader integration with other delivery and monitoring tools, so traceability can cover a wider operational scope.

Broader Implications

Although MES-X was developed in one industrial context, the approach is transferable. Many software teams face the same challenge of connecting planning artifacts with code evidence before release decisions.

The combination used here, graph-based traversal, typed contracts, progressive-loading UI patterns, and decision-focused summaries, can be reused for release governance, impact analysis, and audit-oriented traceability in other environments.

Final Statement

This internship delivered more than a prototype. It delivered a working, modular platform that improves how dependency analysis is performed in practice. MES-X now provides a clearer and more reliable release-preparation workflow, while remaining extensible for future needs. The result is a strong foundation for continued operational use and gradual evolution.